BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN MÔI TRƯỜNG TRANG TRẠI THÔNG MINH

TÊN LỚP HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IOT

MÃ HỌC PHẦN: 2025-2026.2.TIN4024.009

GIẢNG VIÊN HƯỚNG DẪN: TS. LÊ HỮU BÌNH

Huế, Ngày 08 Tháng 04 Năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc đến Khoa Công nghệ Thông tin – Trường Đại học Khoa học – Đại học Huế, nơi đã tạo điều kiện thuận lợi và môi trường học tập tuyệt vời để em có cơ hội tham gia và hoàn thành bài tiểu luận này. Sự hỗ trợ từ phía nhà trường đã giúp em không chỉ phát triển kiến thức chuyên môn mà còn rèn luyện được những kỹ năng cần thiết để chuẩn bị cho tương lai.

Đặc biệt, em xin bày tỏ lòng biết ơn chân thành đến TS. Lê Hữu Bình, người đã tận tình hướng dẫn, chia sẻ kinh nghiệm và truyền đạt những kiến thức quý báu trong suốt quá trình em thực hiện bài tiểu luận. Những lời chỉ bảo tận tình và sự quan tâm của thầy đã giúp em vượt qua những khó khăn và hoàn thành bài tiểu luận một cách tốt nhất. Em cảm thấy may mắn và trân quý những lời khuyên, sự giúp đỡ từ thầy, người đã luôn kiên nhẫn đồng hành cùng em qua từng bước đi.

Bên cạnh đó, em xin gửi lời tri ân đến toàn thể quý thầy cô của Khoa Công nghệ Thông tin – Trường Đại học Khoa học – Đại học Huế, những người đã không quản ngại khó khăn, luôn tận tâm truyền đạt kiến thức, dạy bảo và hỗ trợ em trong suốt thời gian học tập tại trường. Sự tận tụy và cống hiến của quý thầy cô không chỉ giúp em đạt được thành tích học tập mà còn giúp em nhận thức rõ ràng về con đường sự nghiệp mà em đang theo đuổi.

Xin trân trọng cảm ơn!

Huế, tháng 04 năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

Ký hiệu	Giải thích
API	Application Programming Interface — Giao diện lập trình ứng dụng
DHT	Digital Humidity and Temperature — Cảm biến nhiệt độ và độ ẩm số
GPIO	General Purpose Input/Output — Chân vào/ra đa mục đích
IoT	Internet of Things — Internet vạn vật
JSON	JavaScript Object Notation — Định dạng chuỗi dữ liệu khóa-giá trị
LED	Light-Emitting Diode — Điốt phát quang
MQTT	Message Queuing Telemetry Transport — Giao thức truyền thông pub/sub
MQTT Broker	Phần mềm trung gian nhận và phân phối message theo topic
NRD	Node-RED Dashboard — Giao diện Dashboard của Node-RED
Pub/Sub	Publisher/Subscriber — Mô hình gửi/nhận message qua topic
TLS	Transport Layer Security — Giao thức bảo mật tầng truyền tải

Mục lục

CHƯƠNG 1: ĐẶT VẤN ĐỀ **1**
1.1 Bối cảnh và tính cấp thiết của đề tài 1
1.2 Mục tiêu thiết kế và phạm vi nghiên cứu 1

CHƯƠNG 2: NỘI DUNG THỰC HIỆN **2**
2.1 Thu thập dữ liệu môi trường (DHT22 và độ ẩm đất) 2
2.2 Chỉ báo cục bộ: LED mô phỏng máy bơm và buzzer cảnh báo 2
2.3 Truyền dữ liệu qua MQTT (HiveMQ Cloud, TLS) 3
2.4 Xử lý dữ liệu bằng Node-RED 4
2.5 Hiển thị dữ liệu trên Dashboard 6
2.6 Cảnh báo môi trường 6
2.7 Điều khiển máy bơm từ xa qua MQTT 7
2.8 Duy trì kết nối và đồng bộ trạng thái bơm 7
2.9 Tự động tưới theo ngưỡng độ ẩm đất (Node-RED) 8

CHƯƠNG 3: THIẾT KẾ MẠCH IoT **8**
3.1 Thành phần phần cứng 8
3.2 Kết nối phần cứng (GPIO) 9
3.2.1 Sơ đồ logic (EasyEDA) 10
3.2.2 Mạch vật lý — ảnh mô phỏng Wokwi (tương đương mạch thực tế đã nối) 11
3.3 Giải thích các linh kiện chính 12
3.3.1 Vi điều khiển ESP32 DevKit C v4 12
3.3.2 Cảm biến nhiệt độ và độ ẩm không khí DHT22 13
3.3.3 Biến trở (mô phỏng cảm biến độ ẩm đất) 14
3.3.4 Cơ cấu chấp hành và tín hiệu cảnh báo (LED & Buzzer) 14
3.4 Luồng hoạt động tóm tắt theo chức năng 15

CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG **16**
4.1 Sơ đồ luồng hệ thống (kiến trúc ba tầng) 16
4.2 Luồng giám sát: cảm biến → MQTT → Dashboard 17
4.3 Luồng điều khiển: Dashboard → MQTT → ESP32 (LED mô phỏng bơm) 17
4.4 Luồng đồng bộ trạng thái bơm và giới hạn thời gian chạy 19
4.5 Luồng tưới tự động theo ngưỡng độ ẩm đất (Node-RED) 20
4.6 Luồng duy trì kết nối Wi-Fi và MQTT 20

CHƯƠNG 5: CÀI ĐẶT HỆ THỐNG **20**
5.1 Chuẩn bị công cụ và môi trường phát triển 20
5.2 Khởi tạo sơ đồ mạch mô phỏng và cấu hình phần cứng 21
5.3 Triển khai và vận hành Firmware cho ESP32 21
5.3.1 Cài đặt mã nguồn và thư viện 21
5.3.2 Cấu hình thông số hệ thống 21
5.3.3 Vận hành mô phỏng 22

5.4 Thiết lập hệ thống điều khiển và giao diện Node-RED 22
5.4.1 Lệnh cài đặt nền tảng 22
5.4.2 Triển khai luồng xử lý và giao diện Dashboard 22
5.5 Vận hành và kiểm thử tổng thể 22

CHƯƠNG 6: KẾT QUẢ THỰC HIỆN **24**
6.1 Kết quả mô phỏng phần cứng trên Wokwi 24
6.2 Kết quả truyền nhận dữ liệu qua Serial Monitor 25
6.3 Thiết lập luồng xử lý dữ liệu trên Node-RED 26
6.4 Giao diện giám sát và điều khiển Dashboard 26
6.5 Phân tích luồng hoạt động thực tế 27

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **30**
7.1 Các công việc đã hoàn thành 30
7.2 Công việc chưa thực hiện và hạn chế 30
7.3 Hướng phát triển tiếp theo 30

TÀI LIỆU THAM KHẢO
Phụ lục A. Mã nguồn chương trình ESP32 . . . . . .
Phụ lục B. Mã nguồn Node-RED (JSON Flow) . . . . . .
Phụ lục C. Mã nguồn sơ đồ Wokwi (JSON) . . . . . .

CHƯƠNG 1: ĐẶT VẤN ĐỀ

1.1 Bối cảnh và tính cấp thiết của đề tài

Trong bối cảnh biến đổi khí hậu diễn biến phức tạp và quá trình đô thị hóa diễn ra nhanh chóng, việc áp dụng công nghệ thông minh vào sản xuất nông nghiệp ngày càng trở nên quan trọng và thực tiễn. Hiện nay, phương pháp tưới tiêu thủ công tại các mô hình nhà kính, trang trại vừa và nhỏ, hay quy mô hộ gia đình (sân thượng, ban công) thường bộc lộ nhiều hạn chế, tốn kém thời gian và khó đảm bảo cung cấp nước đúng mức độ, đúng thời điểm. Những khó khăn này đặt ra nhu cầu lớn trong việc ứng dụng Internet of Things (IoT) để chuyển đổi số nông nghiệp.

Việc trang bị một hệ thống có khả năng tự động theo dõi xuyên suốt các thông số thiết yếu (nhiệt độ, độ ẩm không khí, độ ẩm của đất) kết hợp với chức năng cảnh báo sớm và điều khiển bơm tưới tiêu từ xa là một tiện ích không thể thiếu. Tính năng tự động hóa giám sát — publish dữ liệu lên MQTT Cloud Broker — cập nhật lên giao diện trực quan (Dashboard) và thực hiện subscribe các lệnh điều khiển ngược trở lại cho thiết bị phần cứng chính là hướng tiếp cận hiệu quả nhằm giải quyết nhu cầu hiển hiện trên. Đồng thời, đây cũng là hướng đi bám sát với mục đích và nội dung cốt lõi mong đợi của học phần Phát triển Ứng dụng IoT.

1.2 Mục tiêu thiết kế và phạm vi nghiên cứu

Đề tài tập trung thiết kế và triển khai một hệ thống mô phỏng quy trình giám sát và điều khiển trang trại thông minh cỡ nhỏ. Các mục tiêu kỹ thuật cụ thể được giới hạn trong phạm vi thực thi thống nhất giữa Firmware và Flow điều khiển như sau:

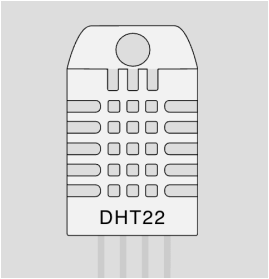
- Đọc nhiệt độ, độ ẩm không khí (DHT22, GPIO 4) và độ ẩm đất mô phỏng bằng biến trở (ADC GPIO 34), chu kỳ 5 giây.
- Publish dữ liệu JSON lên MQTT Broker (HiveMQ Cloud) qua giao thức TLS (cổng 8883), topic `iot/farm/sensor`; đồng thời subscribe để nhận lệnh điều khiển tại topic `iot/farm/control`.

- Điều khiển LED mô phỏng máy bơm (GPIO 5), cảnh báo buzzer (GPIO 18) khi đất khô (ngưỡng raw ADC < 2048 trong code).
- Node-RED Dashboard: gauge, biểu đồ, toast cảnh báo, nút bơm thủ công và chế độ tưới tự động theo **`node-red.json`**.

CHƯƠNG 2: NỘI DUNG THỰC HIỆN

2.1 Thu thập dữ liệu môi trường (DHT22 và độ ẩm đất)

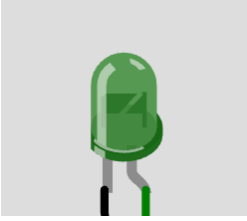
Firmware định kỳ 5 giây (`PUBLISH_INTERVAL`) gọi `readSensor()`: đọc nhiệt độ và độ ẩm không khí qua thư viện DHT trên GPIO `DHT_PIN` (4), đọc độ ẩm đất thô qua `analogRead(SOIL_PIN)` với `SOIL_PIN` = 34 (ADC 12-bit, 0–4095). Nếu DHT trả NaN, giá trị được thay bằng số ngẫu nhiên trong khoảng an toàn để hệ không dừng (**`main.cpp`**). DHT22 đo KK theo giao tiếp 1-wire; độ chính xác tốt hơn DHT11. Potentiometer mô phỏng cảm biến độ ẩm đất: giá trị ADC càng thấp thì trên Dashboard (công thức `(1 - soil/4095)*100`) đất càng khô.



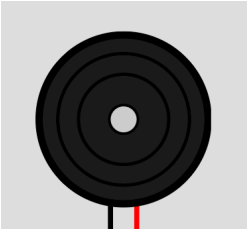
Hình 1: Cảm biến DHT22 đo đạc thông số nhiệt độ và độ ẩm không khí

2.2 Chỉ báo cục bộ: LED mô phỏng máy bơm và buzzer cảnh báo

GPIO `LED_PIN` (5) điều khiển LED cực dương (mô phỏng máy bơm). GPIO `BUZZER_PIN` (18) dùng PWM qua `ledc` trong hàm `toneBuzzer()`: khi `soilRaw < 2048`, firmware phát cảnh báo 2 kHz trong 300 ms mỗi chu kỳ đọc (**`main.cpp`**).



Hình 2: Đèn LED mô phỏng máy bơm



Hình 3: Còi Buzzer phát âm thanh cảnh báo

2.3 Truyền dữ liệu qua MQTT (HiveMQ Cloud, TLS)

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông nhẹ, được thiết kế đặc biệt cho các thiết bị IoT có băng thông thấp, năng lượng hạn chế hoặc kết nối không ổn định. MQTT hoạt động theo mô hình publisher/subscriber (pub/sub), trong đó thiết bị gửi (publisher) thực hiện publish dữ liệu lên một topic trên MQTT Broker, và các thiết bị nhận (subscriber) sẽ subscribe vào các topic tương ứng để nhận dữ liệu. Mô hình này tách biệt người gửi và người nhận, giúp hệ thống dễ mở rộng.

MQTT sử dụng cơ chế QoS (Quality of Service) với ba mức: QoS 0 (At most once) — message được publish một lần, không xác nhận; QoS 1 (At least once) — gửi ít nhất một lần, có xác nhận, đảm bảo message đến; QoS 2 (Exactly once) — đảm bảo message đến đúng một lần, phù hợp cho các ứng dụng yêu cầu độ tin cậy cao. Trong đề tài này, QoS 0 được sử dụng cho cả việc publish dữ liệu cảm biến (`iot/farm/sensor`) và điều khiển (`iot/farm/control`), phù hợp với tần suất dữ liệu không quá cao.

HiveMQ Cloud là dịch vụ MQTT Broker dạng cloud do HiveMQ cung cấp, hỗ trợ giao thức bảo mật TLS (Transport Layer Security) trên cổng 8883, đảm bảo dữ liệu truyền giữa thiết bị và broker được mã hóa end-to-end. HiveMQ Cloud có gói miễn phí (Free Tier) cho phép kết nối tối đa 100 thiết bị, đủ cho các ứng dụng demo và học tập.

Cấu hình MQTT trong đề tài:

- Broker: `b1c8f2cc5cd4416fb74b671a407dc0ff.s1.eu.hivemq.cloud`
- Port: **8883** (TLS)
- Protocol: MQTT v3.1.1
- Topic gửi dữ liệu cảm biến: `iot/farm/sensor`

- Topic nhận lệnh điều khiển: `iot/farm/control`

Lưu ý: topic `iot/farm/sensor` và `iot/farm/control` được đặt theo chuẩn để dễ mở rộng. Trên ESP32, client dùng `WiFiClientSecure` với TLS (cổng `MQTT_PORT` = 8883), `PubSubClient` publish/subscribe tương ứng (**`main.cpp`**).

2.4 Xử lý dữ liệu bằng Node-RED

Node-RED là công cụ lập trình flow-based; dữ liệu từ ESP32 được mô tả chi tiết trong **`node-red.json`**. Message MQTT từ topic `iot/farm/sensor` vào node **mqtt in** rồi phân nhánh song song.

Nhánh thứ nhất (**Gauge nhiệt độ**): message được chuyển qua node function trích giá trị `msg.payload.temperature`, chuyển thành số thực, rồi truyền tới node **ui_gauge** để hiển thị kim đồng hồ đo nhiệt độ với ba vùng màu (xanh: bình thường, vàng: trung gian, đỏ: nguy hiểm).

Nhánh thứ hai (**Gauge độ ẩm không khí**): tương tự, trích giá trị `msg.payload.humidity`, truyền tới node **ui_gauge** (dạng donut) hiển thị độ ẩm không khí trong nhóm **Độ ẩm KK**.

Nhánh thứ ba (**Gauge độ ẩm đất**): trích giá trị `msg.payload.soil` từ potentiometer (raw ADC 0–4095), chuyển đổi thành phần trăm `Math.round((1 - soil / 4095) * 100)`, truyền tới node **ui_gauge** dạng donut với ba vùng màu (đỏ: khô, vàng: trung bình, xanh: ẩm).

Nhánh thứ tư (**Chart nhiệt độ**): trích giá trị nhiệt độ, gán topic `Nhiệt độ °C`, truyền tới node **ui_chart** (line chart) để vẽ đồ thị theo thời gian thực, tự động xóa điểm cũ sau 1 giờ (`removeOlderUnit: 3600`).

Nhánh thứ năm (**Chart độ ẩm KK**): trích giá trị độ ẩm không khí, gán topic `Độ ẩm KK \%`, truyền tới node **ui_chart** để vẽ đồ thị.

Nhánh thứ sáu (**Chart độ ẩm đất**): trích giá trị độ ẩm đất (đã chuyển %), gán topic `Độ ẩm đất \%`, truyền tới node **ui_chart** để vẽ đồ thị.

Nhánh thứ bảy (**Cảnh báo nhiệt độ cao**): switch node kiểm tra `payload.temperature > 35`. Nếu đúng, function node tạo message cảnh báo, truyền tới node **ui_toast** hiển thị thông báo ở góc dưới bên trái Dashboard trong 8 giây.

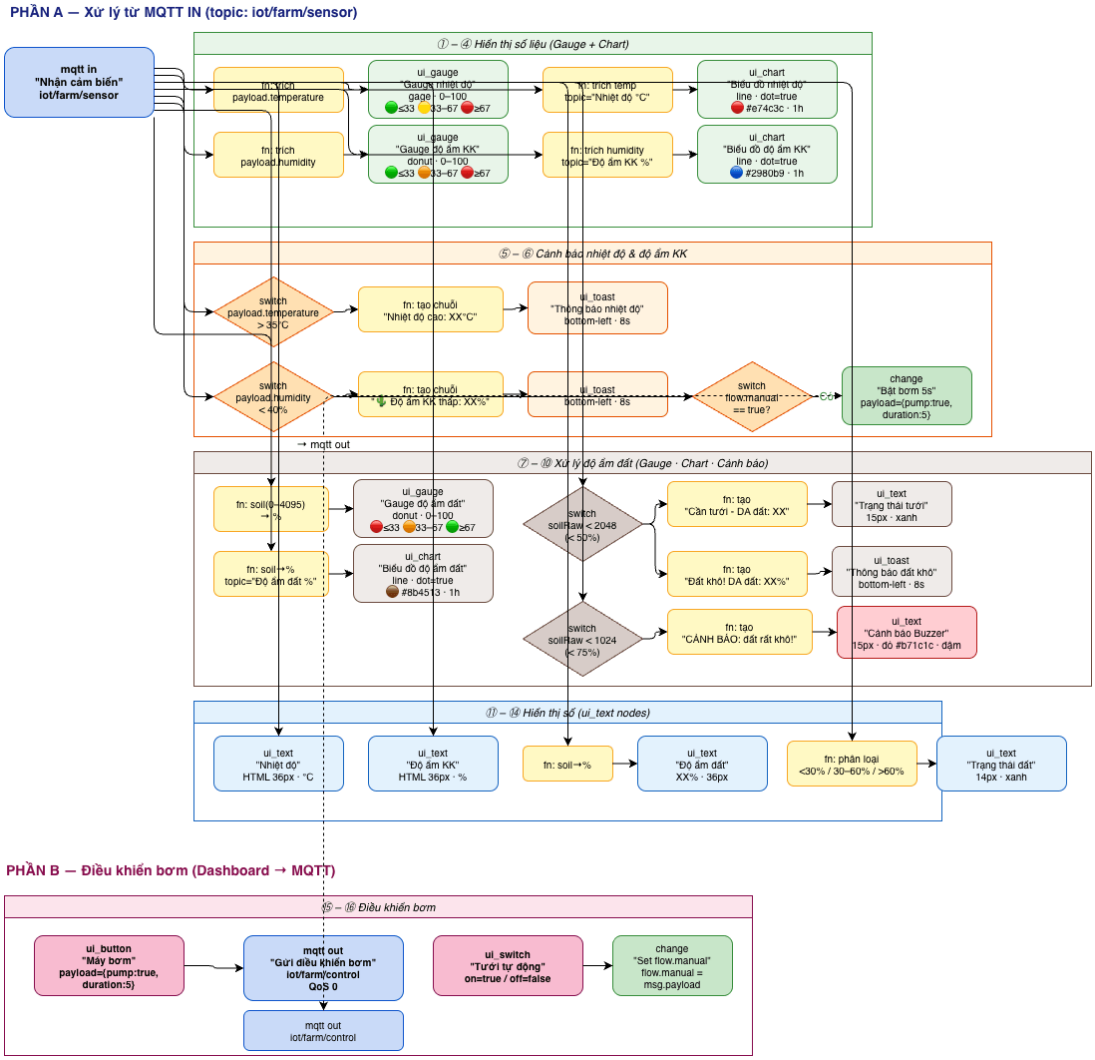
Nhánh thứ tám (**Cảnh báo độ ẩm KK thấp**): switch node kiểm tra `payload.humidity`

< 40. Nếu đúng, function node tạo message cảnh báo kèm gợi ý tưới, truyền tới toast.

Nhánh thứ chín (**Cảnh báo đất khô**): switch node kiểm tra `payload.soil < 2048`. Nếu đúng, function node tạo toast thông báo đất khô kèm giá trị phần trăm.

Nhánh thứ mười (**Cảnh báo nguy hiểm — đất rất khô**): switch node kiểm tra `payload.soil < 1024` (dưới 25% độ ẩm), hiển thị cảnh báo đỏ trên Dashboard (bổ sung cho nhánh 9; buzzer trên ESP32 trong **`main.cpp`** chỉ theo ngưỡng `soilRaw < 2048`, không tách thêm ngưỡng 1024).

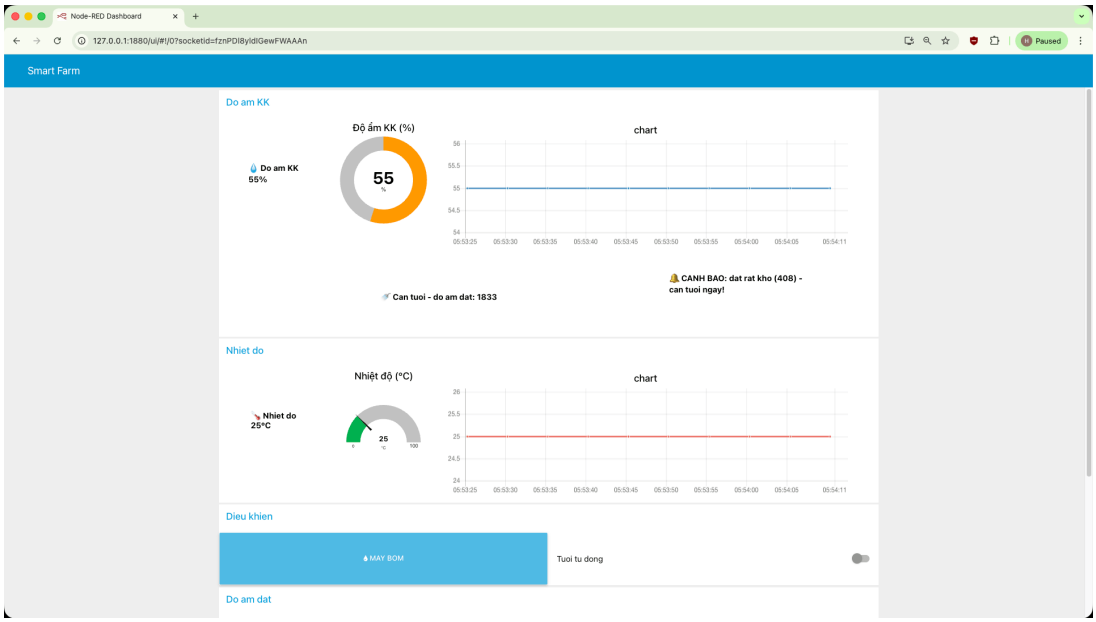
Nhánh thứ mười một (**Tưới tự động**): switch node kiểm tra độ ẩm đất < 2048 đồng thời `flow.manual == true` (switch *Tưới tự động* trên Dashboard gán biến flow tên `manual` — trùng tên với file **`node-red.json`**). Nếu thỏa điều kiện, change node tạo cấu trúc JSON `\{"pump":true,"duration":5\}` và publish thông qua node **mqtt out** đến topic `iot/farm/control`.



Hình 4: Sơ đồ flow xử lý dữ liệu trên Node-RED

2.5 Hiển thị dữ liệu trên Dashboard

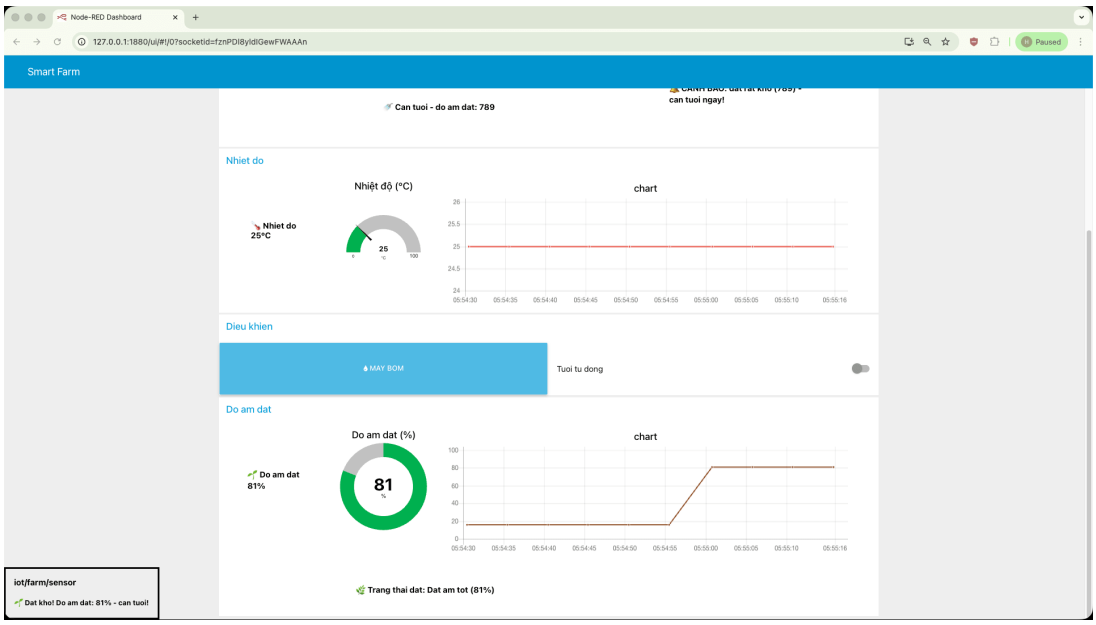
Node-RED Dashboard cung cấp các widget: gauge (nhiệt độ, độ ẩm KK, độ ẩm đất), chart đường theo thời gian thực, text trạng thái, nút **Máy bơm**, switch **Tưới tự động**, toast cảnh báo. Giao diện truy cập tại `http://localhost:1880/ui` (hoặc `http://127.0.0.1:1880/ui`).



Hình 5: Giao diện tổng quan hiển thị dữ liệu trên Node-RED Dashboard

2.6 Cảnh báo môi trường

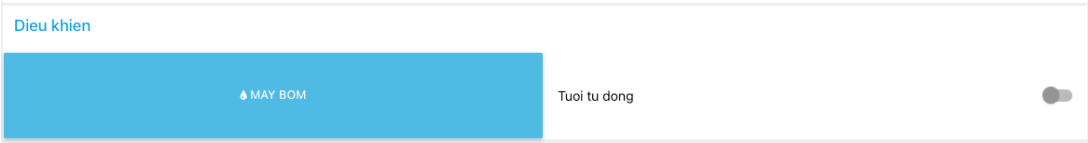
Trên Dashboard: toast khi `temperature > 35`, khi `humidity < 40`, khi `soil < 2048`, và cảnh báo đỏ khi `soil < 1024` (**`node-red.json`**). Trên ESP32: buzzer chỉ phụ thuộc `soilRaw < 2048` trong `loop()` (không có ngưỡng 1024 riêng trên firmware).



Hình 6: Giao diện cảnh báo môi trường trên Node-RED Dashboard

2.7 Điều khiển máy bơm từ xa qua MQTT

Hệ thống cung cấp khả năng điều khiển thiết bị chấp hành thông qua các bản tin định hướng từ máy chủ. Cụ thể, thiết bị ESP32 thực hiện subscribe vào topic điều khiển `iot/farm/control` để chờ nhận lệnh. Khi nhận được một gói tin JSON chứa chỉ thị kích hoạt máy bơm, bộ vi xử lý sẽ tiến hành parse dữ liệu và thực thi việc bật cơ cấu chấp hành (mô phỏng bằng đèn LED). Để đảm bảo an toàn và tiết kiệm tài nguyên, hệ thống tích hợp cơ chế tự động ngắt sau một khoảng thời gian vận hành dự kiến. Quy trình này giúp duy trì sự đồng bộ giữa trạng thái hiển thị trên Dashboard và hoạt động thực tế của thiết bị, đồng thời bảo vệ hệ thống khỏi các lỗi vận hành kéo dài không mong muốn.



Hình 7: Giao diện nút nhấn và công tắc điều khiển máy bơm từ xa

2.8 Duy trì kết nối và đồng bộ trạng thái bơm

Hệ thống được thiết kế để tự động giám sát và duy trì tính ổn định của các kết nối mạng. Trong vòng lặp chính của chương trình, vi điều khiển liên tục kiểm tra trạng thái của

Wi-Fi và giao thức MQTT. Nếu phát hiện sự cố ngắt kết nối, quy trình tái lập liên kết sẽ được thực hiện định kỳ cho đến khi thông tin giao tiếp được khôi phục. Đồng thời, cơ chế giám sát thời gian vận hành máy bơm cũng được thực thi song song để đảm bảo thiết bị tự động ngắt sau một khoảng thời gian quy định, giữ cho trạng thái thực tế luôn khớp với lệnh điều khiển đã gửi.

2.9 Tự động tưới theo ngưỡng độ ẩm đất (Node-RED)

Hệ thống tích hợp logic điều khiển tự động dựa trên sự kết hợp giữa các thông số cảm biến thực tế và thiết lập từ người dùng. Khi chế độ tưới tự động được kích hoạt trên Dashboard, hệ thống sẽ liên tục đánh giá giá trị độ ẩm đất đo được. Nếu độ ẩm giảm xuống dưới ngưỡng khô hạn đã thiết lập, Node-RED sẽ tự động tạo và publish các payload điều khiển qua giao thức MQTT để kích hoạt máy bơm nước tưới bù độ ẩm, đảm bảo duy trì môi trường lý tưởng cho cây trồng.

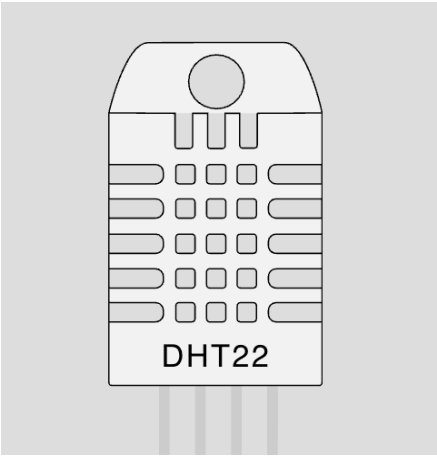
CHƯƠNG 3: THIẾT KẾ MẠCH IoT

3.1 Thành phần phần cứng

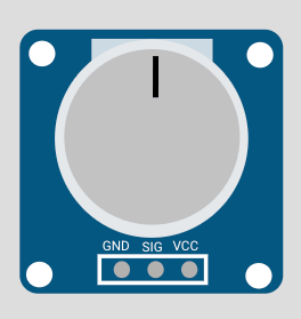
Hệ thống sử dụng các linh kiện điện tử cơ bản và phổ biến, được lựa chọn nhằm tối ưu chi phí nhưng vẫn đảm bảo độ tin cậy trong việc thu thập dữ liệu và cảnh báo. Các thành phần chính bao gồm:

- **Vi điều khiển trung tâm:** ESP32 DevKit C v4 đóng vai trò bộ não xử lý, kết nối mạng Wi-Fi và giao tiếp MQTT.
- **Cảm biến môi trường:** Cảm biến DHT22 dùng để đo nhiệt độ và độ ẩm không khí với sai số thấp.
- **Cảm biến đo lường (mô phỏng):** Biến trở (Potentiometer) được sử dụng để giả lập tín hiệu điện áp tương tự (ADC) của cảm biến đo độ ẩm đất trong thực tế.
- **Cơ cấu chấp hành và cảnh báo:** Đèn LED (kèm điện trở hạn dòng) giả lập trạng thái hoạt động của máy bơm nước. Còi loa (Buzzer) điều khiển bằng xung PWM nhằm phát âm thanh báo động cường độ cao khi đất quá khô.

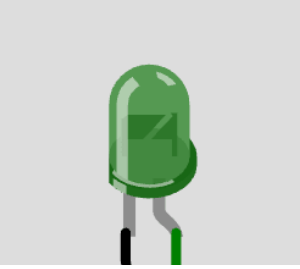
Hình 8: Vi điều khiển ESP32



Hình 9: Cảm biến DHT22



Hình 10: Biến trở



Hình 11: Đèn LED

Hình 12: Còi Buzzer

3.2 Kết nối phần cứng (GPIO)

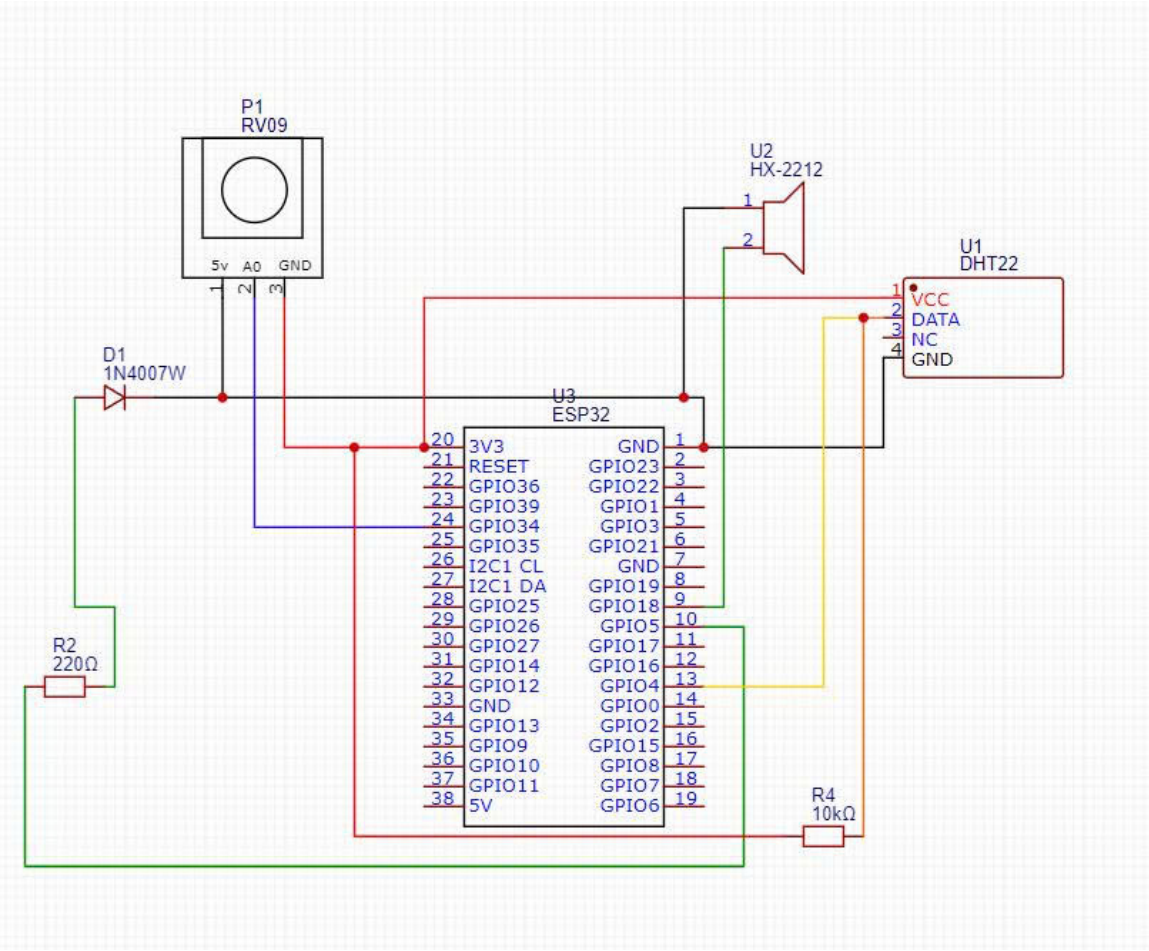
Bảng 1 tóm tắt nối dây.

Bảng 1: Bảng kết nối chân GPIO trên ESP32

Chân ESP32	**Nối đến**	**Chức năng**
3V3	DHT22 VCC, Potentiometer VCC	Nguồn 3.3V cho cảm biến
GND	DHT22 GND, LED Cathode, Potentiometer GND, Buzzer GND	Chân nối đất chung
GPIO 4	DHT22 DATA	Bus dữ liệu 1-wire (nhiệt độ + độ ẩm KK)
GPIO 5	LED Anode (qua R220)	Điều khiển bật/tắt LED (mô phỏng bơm)
GPIO 18	Buzzer (chân 2)	Phát âm thanh cảnh báo khi đất khô
GPIO 34	Potentiometer SIG	Đọc giá trị độ ẩm đất (ADC, chỉ đọc)

3.2.1 Sơ đồ logic (EasyEDA)

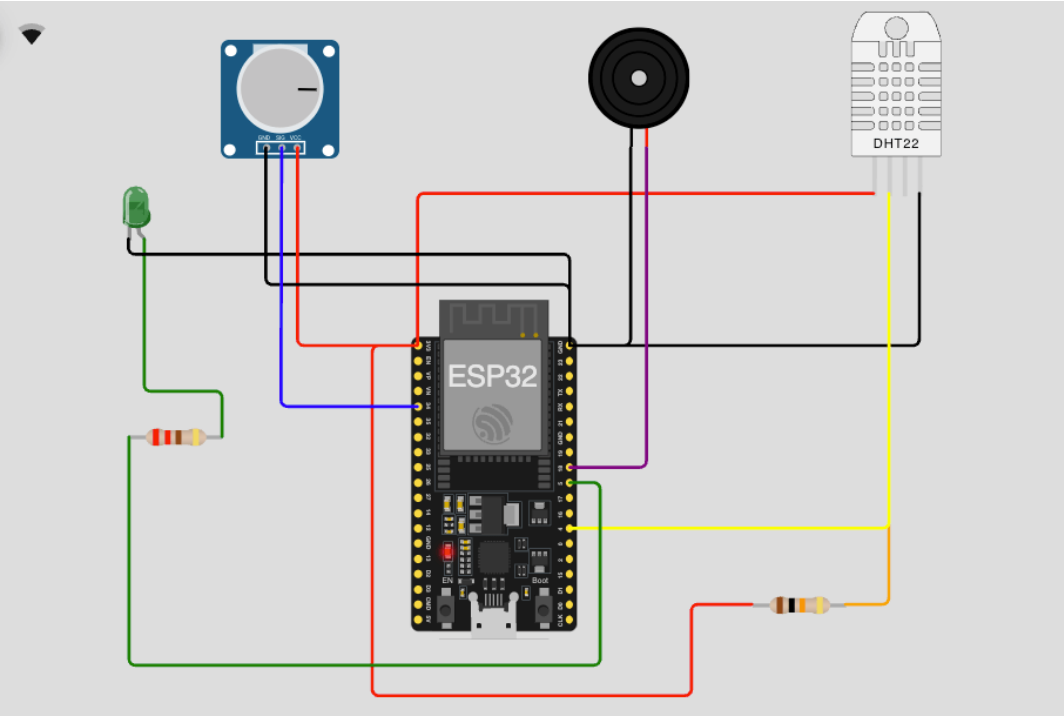
Hình 13.



Hình 13: Sơ đồ nguyên lý phần cứng (EasyEDA): ESP32, DHT22, đất, relay/bơm (mở rộng), buzzer, OLED (mở rộng) — phần lõi đã code nằm ở DHT22, ADC đất, LED, buzzer

3.2.2 Mạch vật lý — ảnh mô phỏng Wokwi (tương đương mạch thực tế đã nối)

Hình 14.



Hình 14: Mạch mô phỏng Wokwi: ESP32 DevKit C v4 + DHT22 + LED + Potentiometer + Buzzer (khớp `diagram.json`)

3.3 Giải thích các linh kiện chính

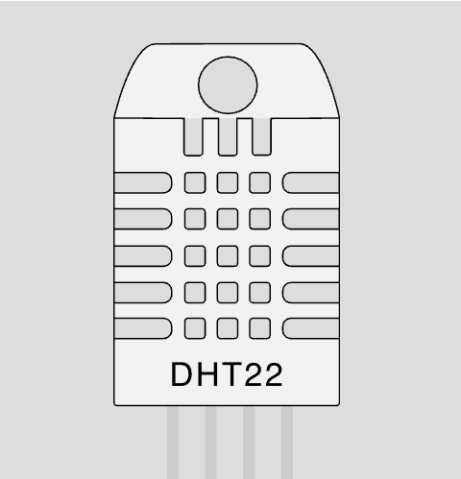
3.3.1 Vi điều khiển ESP32 DevKit C v4

ESP32 là một hệ thống trên chip (SoC) tích hợp Wi-Fi và Bluetooth, được xây dựng trên nền vi xử lý Xtensa LX6 dual-core hiệu năng cao. Với bộ chuyển đổi tín hiệu tương tự sang số (ADC) 12-bit có độ phân giải cao, ESP32 có khả năng đọc chính xác các tín hiệu điện áp từ cảm biến analog. Trong hệ thống này, ESP32 đóng vai trò trung tâm điều phối: thu thập dữ liệu từ cảm biến, xử lý tín hiệu và duy trì kết nối liên tục lên đám mây qua giao thức MQTT bảo mật.

Hình 15: Mạch vi điều khiển ESP32 DevKit C v4

3.3.2 Cảm biến nhiệt độ và độ ẩm không khí DHT22

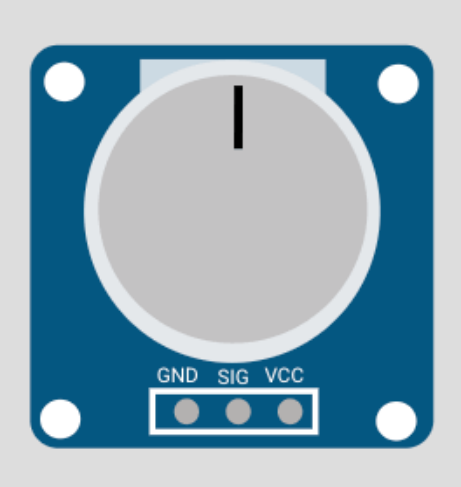
DHT22 là cảm biến đo môi trường sử dụng giao tiếp số 1-wire, cho phép đo đồng thời nhiệt độ (độ chính xác ±0.5°C) và độ ẩm không khí (độ chính xác ±2–5%). So với thế hệ trước (DHT11), DHT22 cải thiện đáng kể về phạm vi đo và độ phân giải, phù hợp cho các ứng dụng nông nghiệp yêu cầu theo dõi môi trường chính xác.



Hình 16: Cảm biến DHT22 đo nhiệt độ và độ ẩm không khí

3.3.3 Biến trở (mô phỏng cảm biến độ ẩm đất)

Trong phạm vi mô phỏng của đề tài, biến trở (Potentiometer) được sử dụng thay thế cho cảm biến đo độ ẩm đất chuyên dụng. Khi xoay trục biến trở, điện trở thay đổi làm thay đổi mức điện áp trên chân tín hiệu, từ đó mô phỏng dải giá trị từ đất khô hoàn toàn đến đất bão hòa nước. Kết quả đọc được qua bộ ADC sẽ được quy đổi sang phần trăm độ ẩm để hiển thị trực quan trên Dashboard.

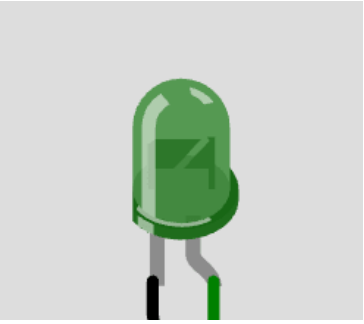


Hình 17: Biến trở xoay dùng mô phỏng tín hiệu độ ẩm đất

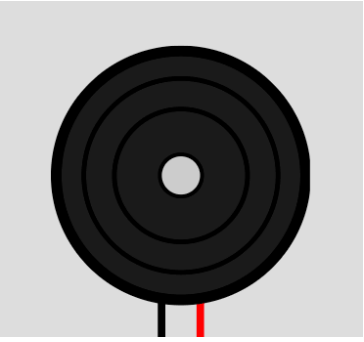
3.3.4 Cơ cấu chấp hành và tín hiệu cảnh báo (LED & Buzzer)

Hệ thống sử dụng hai thiết bị đầu ra để phản ánh trực quan kết quả điều khiển. Đèn LED đóng vai trò giả lập trạng thái hoạt động của máy bơm nước: sáng khi bơm đang vận hành, tắt khi bơm ngừng. Còi sinh âm (Buzzer) được điều khiển bằng xung PWM, phát

tín hiệu âm thanh cường độ cao khi hệ thống phát hiện độ ẩm đất xuống dưới ngưỡng cảnh báo khô hạn, nhằm thông báo kịp thời cho người vận hành.



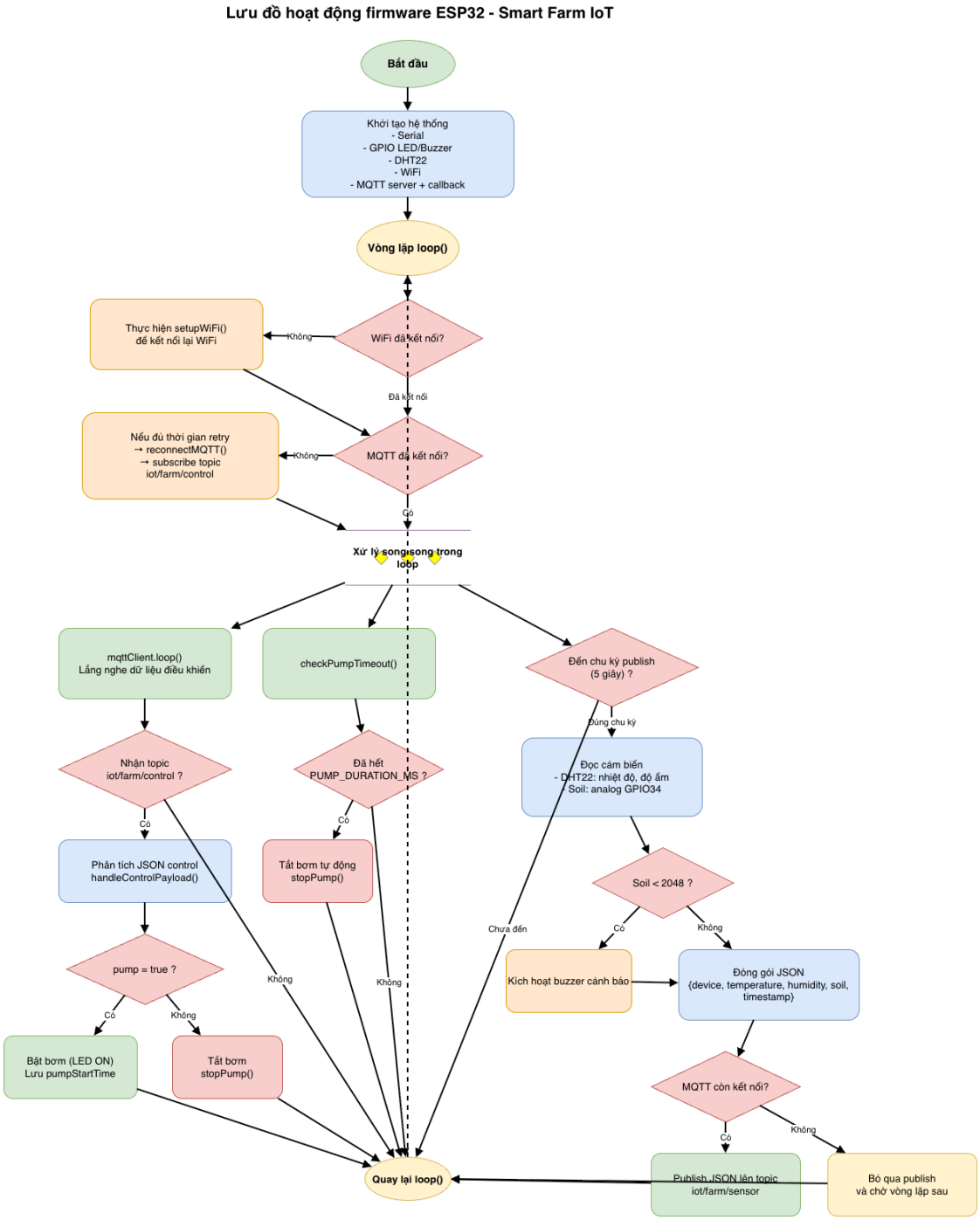
Hình 18: Đèn LED mô phỏng trạng thái máy bơm



Hình 19: Còi Buzzer phát âm thanh cảnh báo

3.4 Luồng hoạt động tóm tắt theo chức năng

Toàn bộ hoạt động của hệ thống diễn ra theo một vòng lặp liên tục: vi điều khiển định kỳ thu thập dữ liệu từ các cảm biến, thực hiện kiểm tra ngưỡng để kích hoạt cảnh báo âm thanh khi cần, sau đó đóng gói kết quả thành bản tin JSON và gửi lên broker MQTT. Song song với đó, hệ thống luôn lắng nghe các lệnh điều khiển từ Dashboard để bật hoặc tắt máy bơm, đồng thời tự động ngắt bơm sau một khoảng thời gian chạy định sẵn nhằm tránh bơm hoạt động quá lâu. Chi tiết về sơ đồ kiến trúc và luồng dữ liệu được trình bày trong Chương 4.



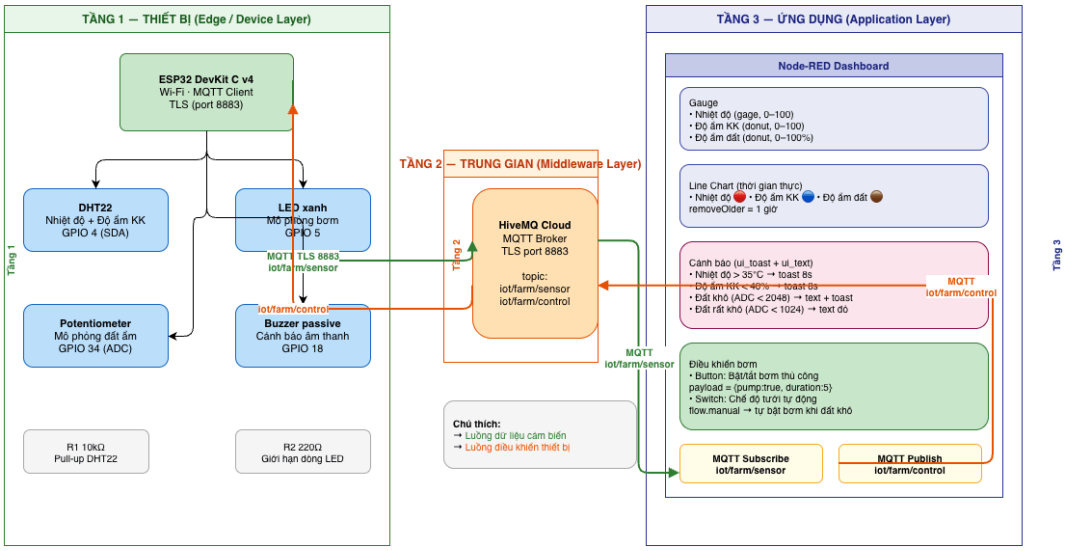
Hình 20: Sơ đồ luồng hoạt động tóm tắt theo chức năng của hệ thống

CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG

4.1 Sơ đồ luồng hệ thống (kiến trúc ba tầng)

Kiến trúc của hệ thống được xây dựng dựa trên mô hình ba tầng tiêu chuẩn của một ứng dụng IoT hiện đại. Tầng dưới cùng là tầng thiết bị, bao gồm vi điều khiển ESP32 kết nối trực tiếp với các cảm biến và cơ cấu chấp hành. Tầng chính giữa là tầng trung gian, sử

dụng dịch vụ Broker của HiveMQ Cloud với giao thức bảo mật TLS để chuyển tiếp dữ liệu một cách an toàn. Trên cùng là tầng ứng dụng với Node-RED Dashboard, đóng vai trò là giao diện trung tâm để người dùng giám sát các thông số môi trường và phát lệnh điều khiển ngược trở lại thiết bị qua các chủ đề (topic) đã được định nghĩa thống nhất.



Hình 21: Sơ đồ kiến trúc tổng thể hệ thống giám sát và điều khiển môi trường trang trại thông minh

4.2 Luồng giám sát: cảm biến → MQTT → Dashboard

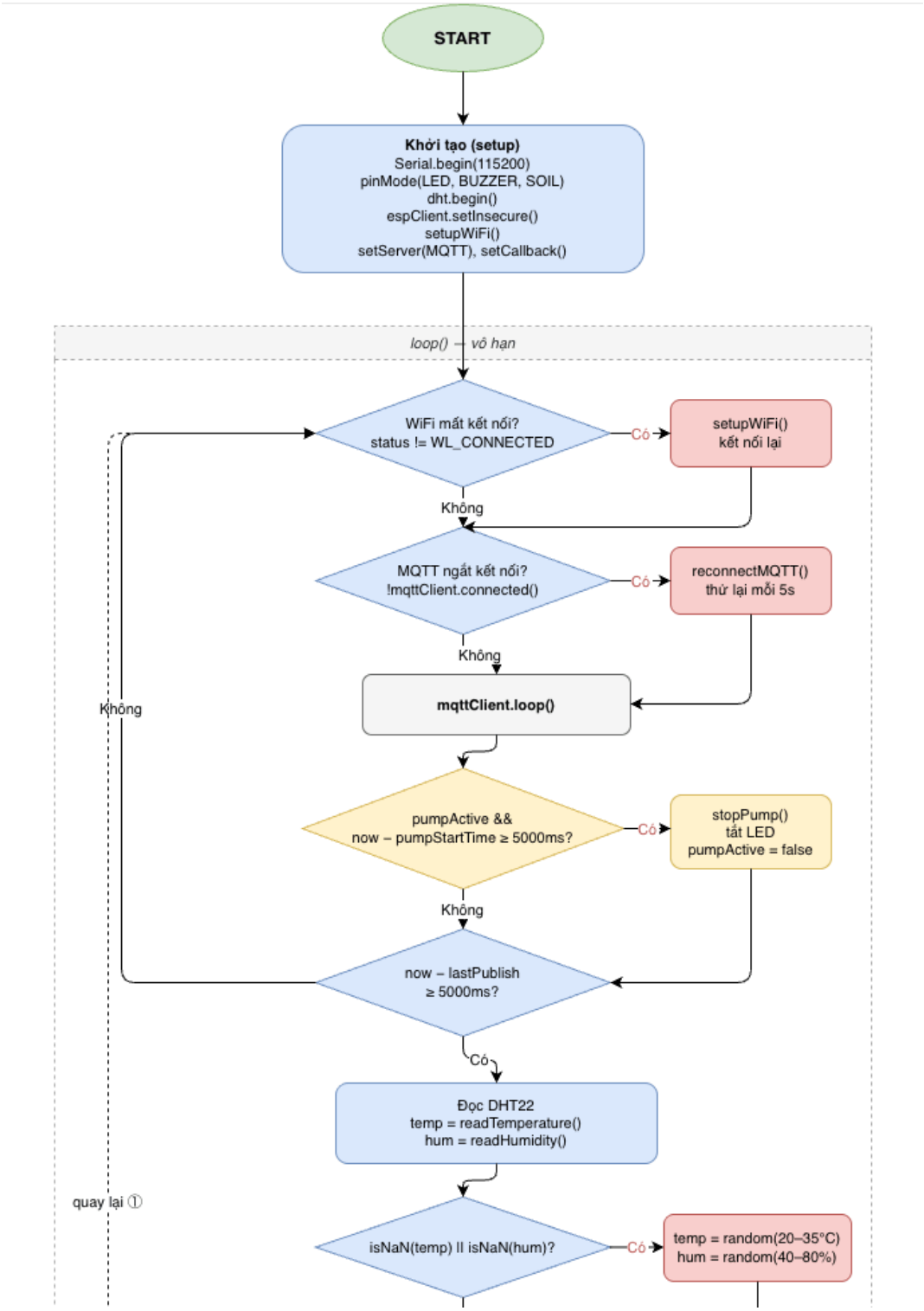
Luồng giám sát được thực hiện theo cơ chế định kỳ: cứ mỗi chu kỳ 5 giây, vi điều khiển đọc dữ liệu từ cảm biến nhiệt độ, độ ẩm không khí và tín hiệu ADC từ biến trở mô phỏng độ ẩm đất. Toàn bộ dữ liệu được đóng gói thành một bản tin JSON có cấu trúc thống nhất rồi gửi lên MQTT Broker (HiveMQ Cloud) qua kết nối bảo mật TLS. Broker sau đó phân phát bản tin tới Node-RED — nơi các luồng xử lý song song phân tích giá trị và cập nhật lên các widget tương ứng trên Dashboard: đồng hồ đo (gauge), biểu đồ theo thời gian thực (line chart) và thông báo cảnh báo (toast notification). Kiến trúc ba tầng của luồng này được minh họa tại Hình 21 (mục 4.1).

4.3 Luồng điều khiển: Dashboard → MQTT → ESP32 (LED mô phỏng bơm)

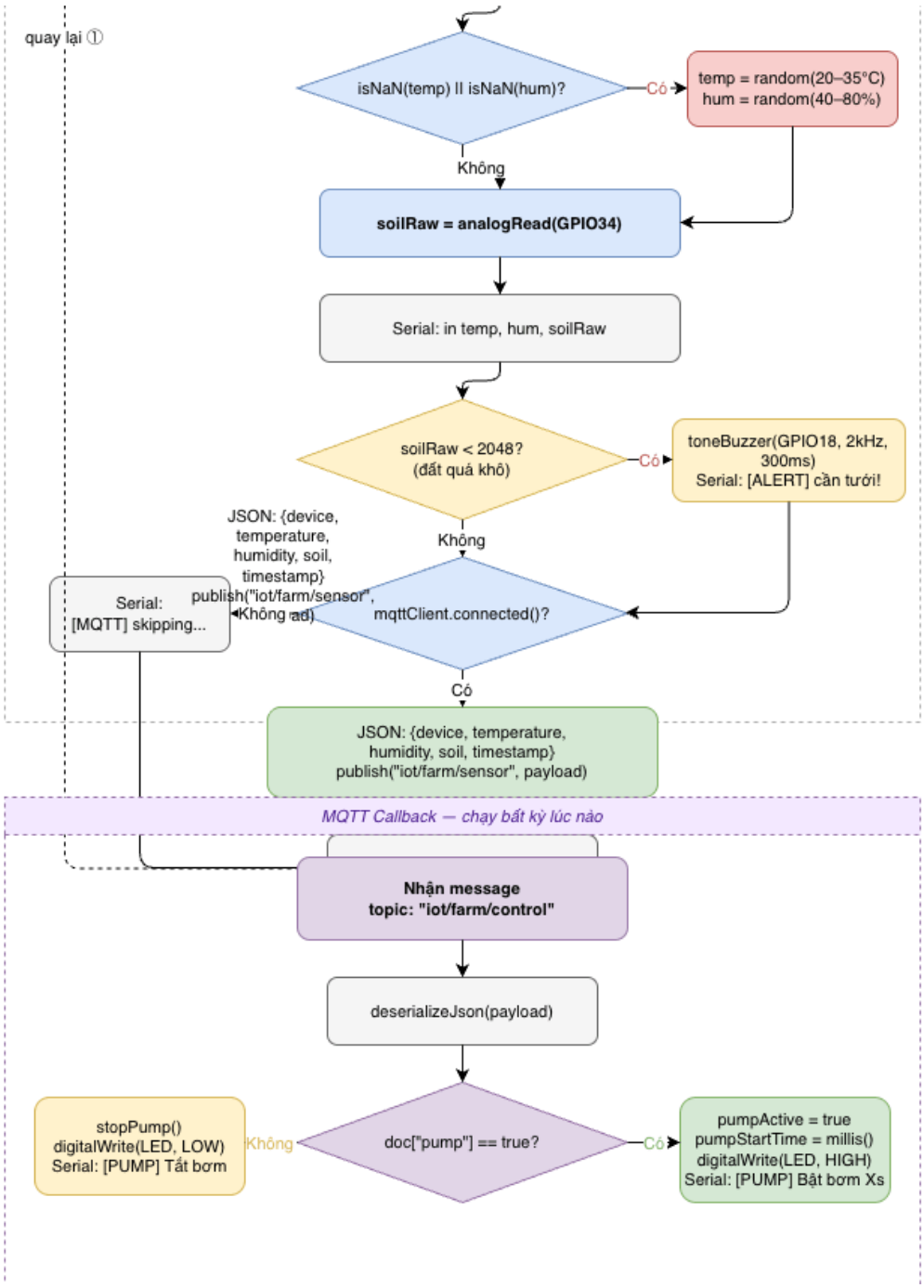
Người dùng bấm nút hoặc kích hoạt tưới tự động trên Dashboard; Node-RED publish JSON xuống `iot/farm/control`; `mqttCallback()` → `handleControlPayload()` bật

LED và ghi `pumpStartTime`.

ESP32 thực hiện chu trình: khởi tạo Serial 115200, Wi-Fi, DHT, GPIO, MQTT TLS, subscribe `iot/farm/control`. Vòng `loop()`: duy trì Wi-Fi/MQTT, `checkPumpTimeout()`, mỗi 5 s đọc cảm biến, buzzer nếu đất khô, `publishSensorData()` nếu broker sẵn sàng.



Hình 22: Lưu đồ thuật toán trên ESP32 (phần trên: `setup()`, `loop()` — Wi-Fi, MQTT, timeout bơm, nhịp 5 giây)



Hình 23: Lưu đồ thuật toán trên ESP32 (phần dưới: đọc cảm biến, buzzer, publish và `mqttCallback` nhận `iot/farm/control`)

4.4 Luồng đồng bộ trạng thái bơm và giới hạn thời gian chạy

Để tránh tình trạng máy bơm hoạt động không kiểm soát, hệ thống triển khai cơ chế giám sát thời gian chạy tự động. Ngay khi nhận lệnh kích hoạt bơm từ Dashboard, vi điều khiển ghi lại thời điểm bắt đầu và theo dõi thời gian đã trôi qua. Khi thời gian vận hành đạt ngưỡng giới hạn định sẵn (5 giây), hệ thống tự động ngắt bơm và cập nhật trạng

thái đèn LED tương ứng — đảm bảo hành vi thực tế của thiết bị luôn đồng bộ với trạng thái hiển thị trên giao diện điều khiển.

4.5 Luồng tưới tự động theo ngưỡng độ ẩm đất (Node-RED)

Chế độ tưới tự động được kích hoạt dựa trên hai điều kiện đồng thời: giá trị độ ẩm đất đo được thấp hơn ngưỡng cảnh báo khô hạn, và người dùng đã bật chế độ tự động trên Dashboard. Khi cả hai điều kiện này được thỏa mãn, Node-RED tự động phát lệnh kích hoạt bơm mà không cần can thiệp thủ công. Cơ chế này hoàn toàn dựa trên giá trị cảm biến thực tế, không phụ thuộc vào lịch thời gian hay đồng hồ hệ thống.

4.6 Luồng duy trì kết nối Wi-Fi và MQTT

Tính ổn định của hệ thống IoT phụ thuộc lớn vào khả năng tự phục hồi kết nối mạng. Vi điều khiển liên tục kiểm tra trạng thái kết nối Wi-Fi trong mỗi vòng lặp; nếu phát hiện mất kết nối, hệ thống tự động thực hiện kết nối lại mà không làm gián đoạn các tác vụ khác. Tương tự, nếu phiên kết nối MQTT bị ngắt, hệ thống định kỳ thử kết nối lại tối đa mỗi 5 giây — đảm bảo luồng dữ liệu cảm biến và các lệnh điều khiển không bị gián đoạn lâu dài.

CHƯƠNG 5: CÀI ĐẶT HỆ THỐNG

5.1 Chuẩn bị công cụ và môi trường phát triển

Trước khi tiến hành cài đặt, người dùng cần chuẩn bị các phần mềm và môi trường nền tảng sau để đảm bảo hệ thống vận hành ổn định:

- **Visual Studio Code:** Trình soạn thảo mã nguồn chính, tải tại `https://code.visualstudio.com/`.
- **PlatformIO IDE:** Tiện ích mở rộng trong VS Code dùng để quản lý dự án và thư viện cho ESP32, thông tin tại `https://platformio.org/`.
- **Node.js:** Nền tảng thực thi JavaScript phía máy chủ, tải phiên bản LTS tại `https://nodejs.org/`.

- **Node-RED:** Công cụ lập trình luồng (flow-based), hướng dẫn cài đặt tại https://nodered.org/docs/getting-started/local.

5.2 Khởi tạo sơ đồ mạch mô phỏng và cấu hình phần cứng

Trong đề tài này, việc kiểm thử phần cứng được thực hiện thông qua nền tảng mô phỏng trực tuyến Wokwi (https://wokwi.com/). Quy trình thiết lập như sau:

1. Truy cập Wokwi và khởi tạo một dự án ESP32 mới.
2. Sao chép toàn bộ nội dung mã nguồn JSON tại **Phụ lục C (Sơ đồ mạch Wokwi)**.
3. Mở thẻ `diagram.json` trong trình soạn thảo của Wokwi và dán nội dung đã sao chép vào.
4. Hệ thống sẽ tự động khởi tạo các linh kiện (ESP32, DHT22, Biến trở, LED, Buzzer) và các kết nối dây tương ứng theo đúng thiết kế của đề tài.

5.3 Triển khai và vận hành Firmware cho ESP32

5.3.1 Cài đặt mã nguồn và thư viện

Người dùng thực hiện sao chép toàn bộ mã nguồn chương trình tại **Phụ lục A (Mã nguồn ESP32)** và dán vào tệp `main.cpp` trong dự án PlatformIO hoặc thẻ `sketch.ino` trên Wokwi. Các thư viện như *DHT sensor library*, *PubSubClient* và *ArduinoJson* sẽ được hệ thống tự động nhận diện và quản lý.

5.3.2 Cấu hình thông số hệ thống

Để thiết bị có thể kết nối với hạ tầng đám mây, cần cập nhật các thông số sau trong tệp mã nguồn:

- **Kết nối mạng:** Cập nhật tên và mật khẩu Wi-Fi. (Trên Wokwi mặc định dùng tên dự án và mật khẩu trống hoặc mạng giả lập).
- **Xác thực MQTT:** Cập nhật các thông tin định danh bao gồm địa chỉ máy chủ, tên đăng nhập và mật khẩu truy cập (Broker, Username, Password) sao cho khớp với tài khoản HiveMQ Cloud đã đăng ký.

5.3.3 Vận hành mô phỏng

Vì hệ thống sử dụng môi trường mô phỏng Wokwi, người dùng không cần kết nối thiết bị vật lý qua cổng USB. Chỉ cần nhấn nút **Start Simulation** để bắt đầu chạy chương trình. Toàn bộ quá trình kết nối và dữ liệu cảm biến sẽ được hiển thị trực tiếp qua cửa sổ *Serial Monitor* của trình giả lập.

5.4 Thiết lập hệ thống điều khiển và giao diện Node-RED

5.4.1 Lệnh cài đặt nền tảng

Mở cửa sổ dòng lệnh (Terminal) và thực hiện cài đặt Node-RED cùng giao diện Dashboard thông qua trình quản lý gói npm:

```
npm install -g node-red
npm install -g node-red-dashboard
```

Sau đó, gõ lệnh `node-red` để khởi động dịch vụ và truy cập vào địa trị http://127.0.0.1:1880.

5.4.2 Triển khai luồng xử lý và giao diện Dashboard

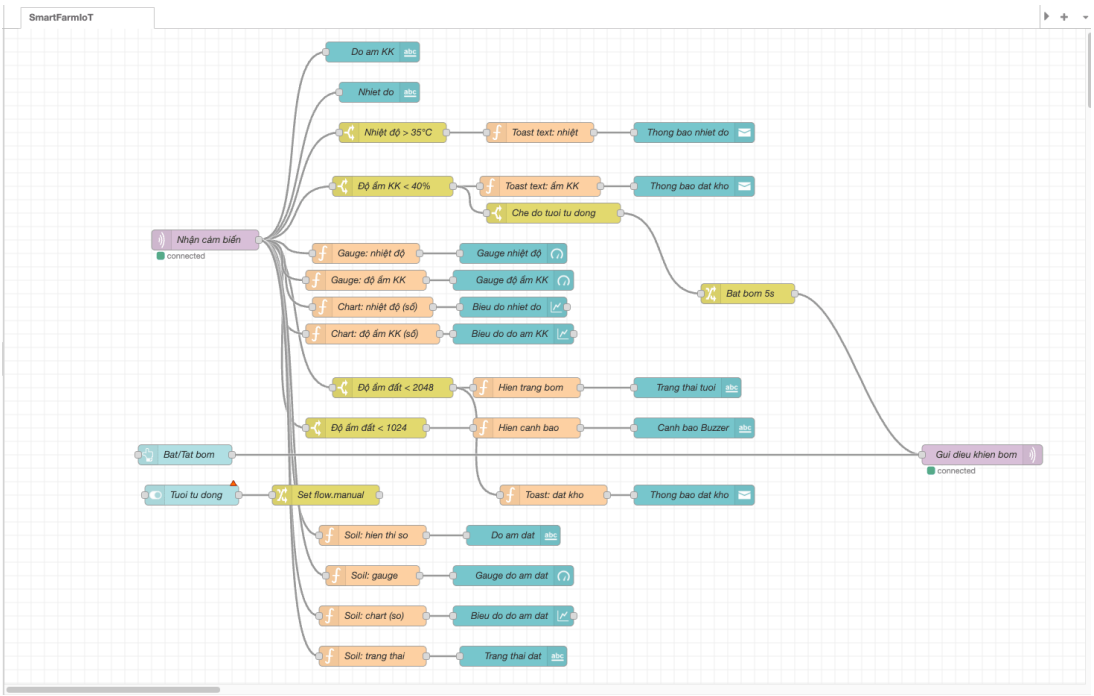
Để thiết lập logic điều khiển và giao diện, người dùng thực hiện:

1. Sao chép mã nguồn JSON tại **Phụ lục B (Mã nguồn Node-RED Flow)**.
2. Tại giao diện Node-RED, chọn **Menu** → **Import** và dán nội dung vào cửa sổ hiện ra.
3. Nhấn **Import** để hệ thống tự động tạo ra các luồng xử lý và giao diện Dashboard.
4. Cập nhật cấu hình node **mqtt-broker** để thiết lập thông tin đăng nhập HiveMQ Cloud tương tự như phía ESP32.

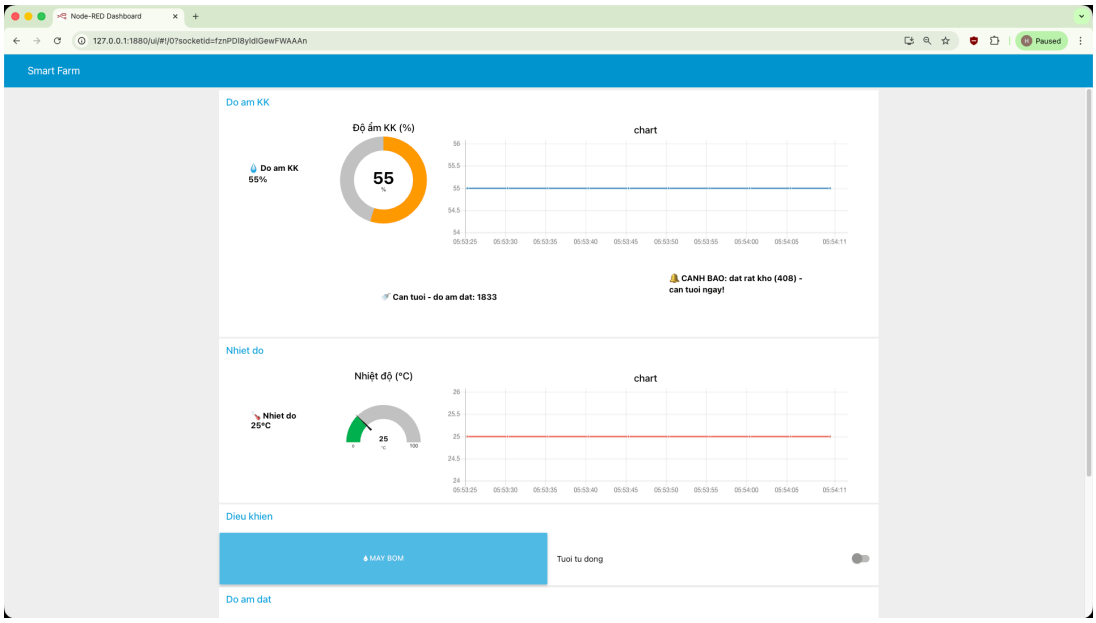
5.5 Vận hành và kiểm thử tổng thể

Sau khi hoàn tất cài đặt, người dùng truy cập Dashboard tại http://127.0.0.1:1880/ui. Hệ thống hoạt động đúng khi các đồng hồ đo hiển thị giá trị nhiệt độ, độ ẩm thực tế

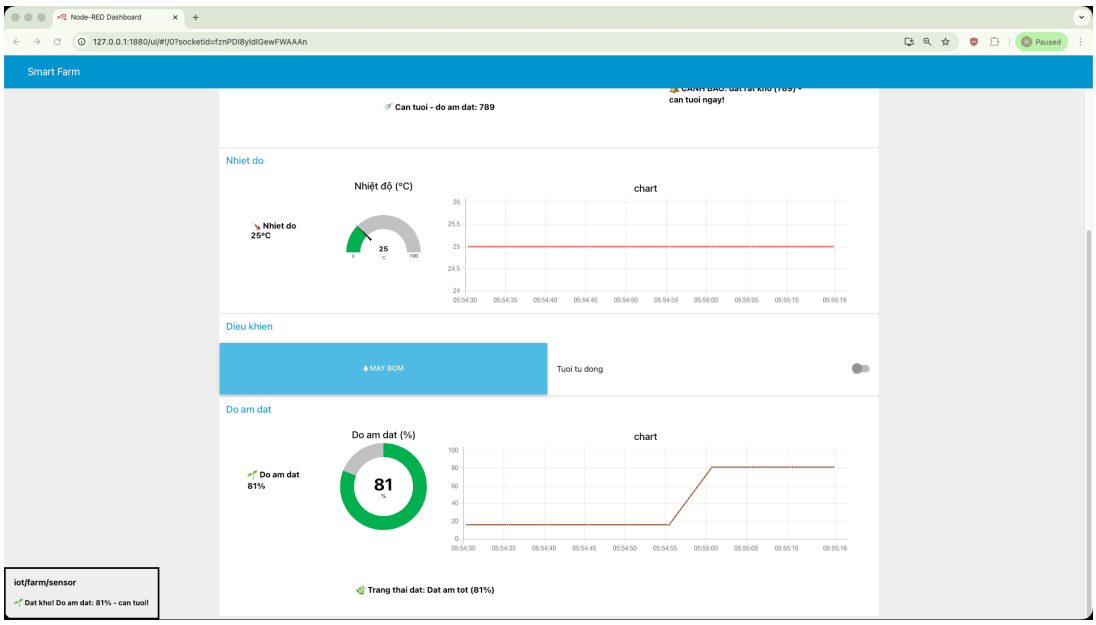
từ Wokwi, và người dùng có thể thực hiện bật/tắt máy bơm từ xa thông qua các nút nhấn trên giao diện.



Hình 24: Giao diện editor Node-RED với flow đầy đủ



Hình 25: Giao diện Dashboard Node-RED (phần 1/2): độ ẩm không khí, nhiệt độ, điều khiển máy bơm và tưới tự động, phần đầu nhóm độ ẩm đất

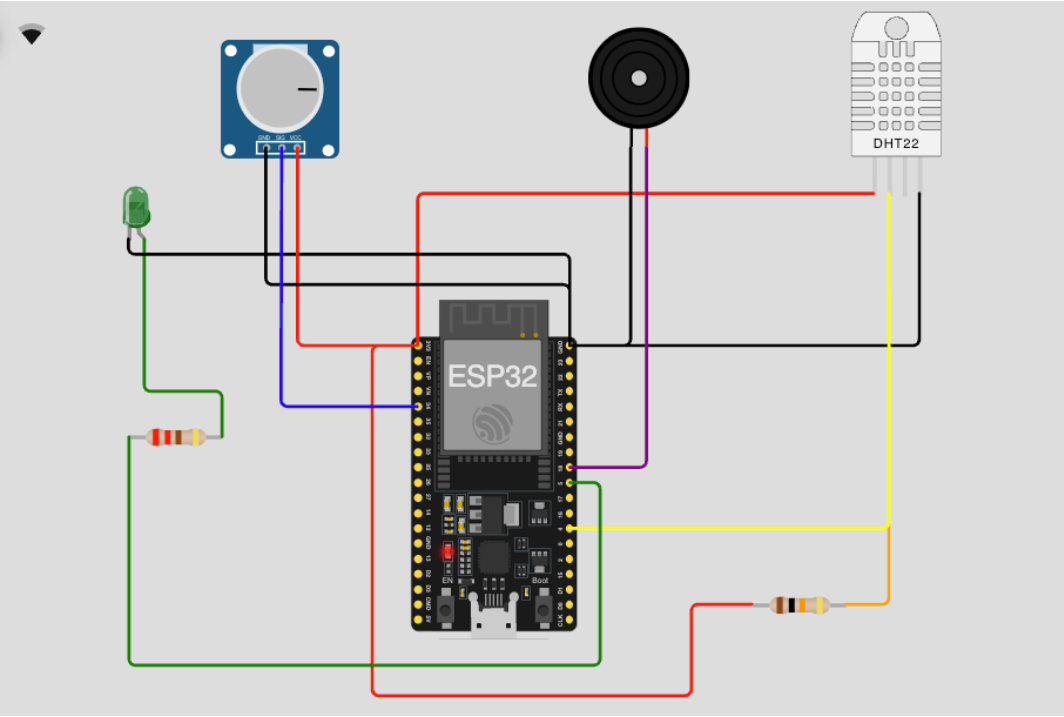


Hình 26: Giao diện Dashboard Node-RED (phần 2/2): nhiệt độ, điều khiển, độ ẩm đất (gauge, biểu đồ line, trạng thái đất và cảnh báo)

CHƯƠNG 6: KẾT QUẢ THỰC HIỆN

6.1 Kết quả mô phỏng phần cứng trên Wokwi

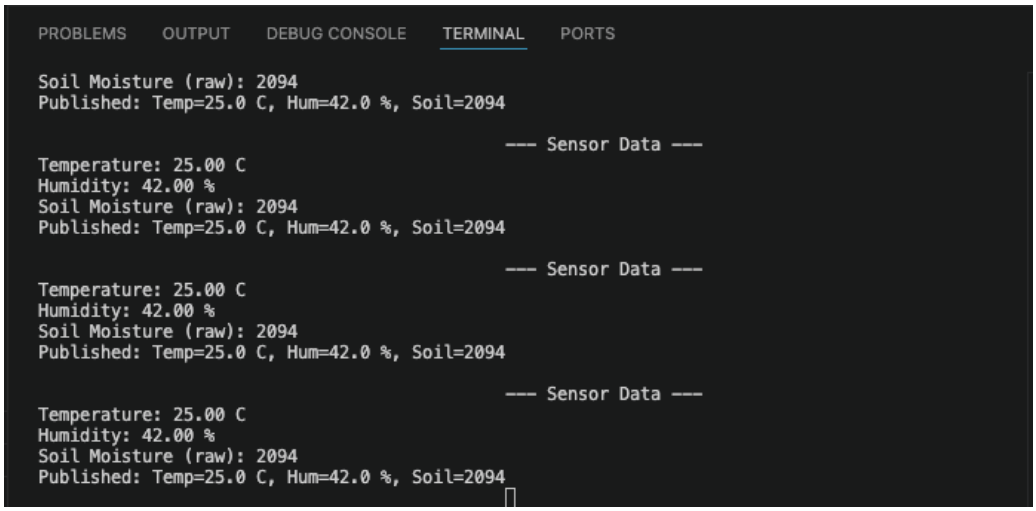
Mạch điện của hệ thống được thực hiện mô phỏng đầy đủ trên nền tảng Wokwi. Hình 27 cho thấy sự kết nối chính xác giữa ESP32 và các linh kiện như cảm biến DHT22, biến trở mô phỏng độ ẩm đất, còi báo động Buzzer và đèn LED giả lập máy bơm.



Hình 27: Mô phỏng sơ đồ kết nối phần cứng thực tế trên môi trường Wokwi

6.2 Kết quả truyền nhận dữ liệu qua Serial Monitor

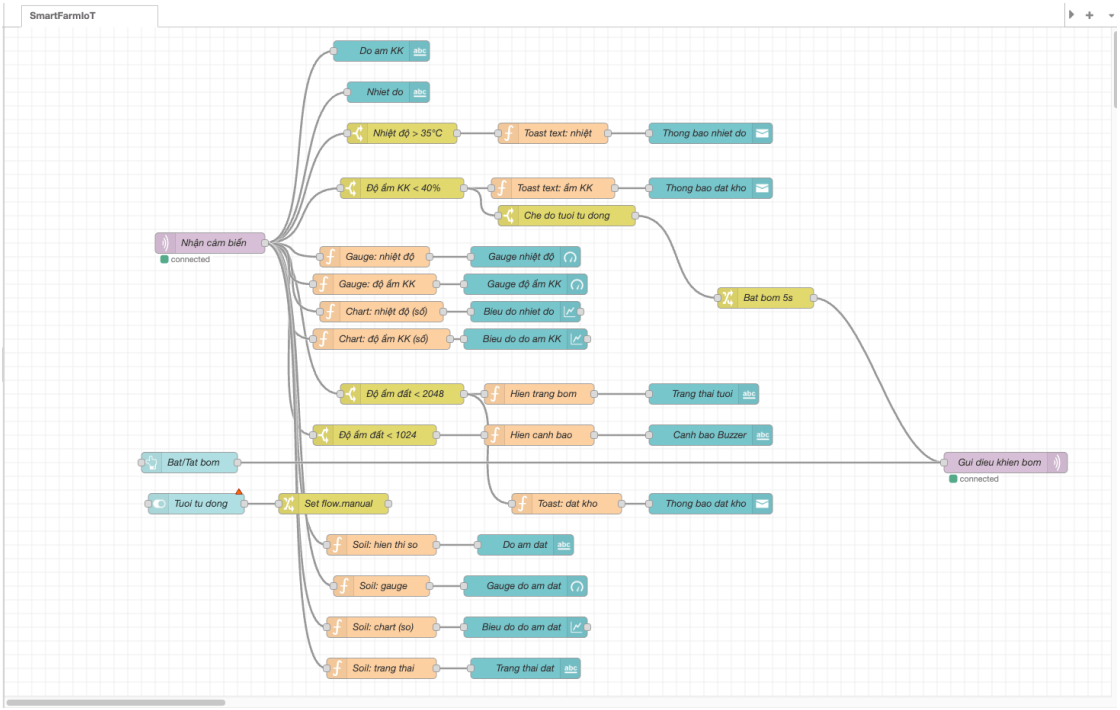
Trong quá trình vận hành, vi điều khiển ESP32 liên tục gửi các thông tin trạng thái hoạt động lên cửa sổ Serial Monitor. Các thông tin bao gồm quá trình kết nối Wi-Fi, trạng thái thiết lập kết nối bảo mật TLS với HiveMQ Cloud và dữ liệu cảm biến thu thập được trước khi đóng gói JSON (Hình 28).



Hình 28: Dữ liệu log hoạt động và trạng thái kết nối MQTT trên Serial Monitor

6.3 Thiết lập luồng xử lý dữ liệu trên Node-RED

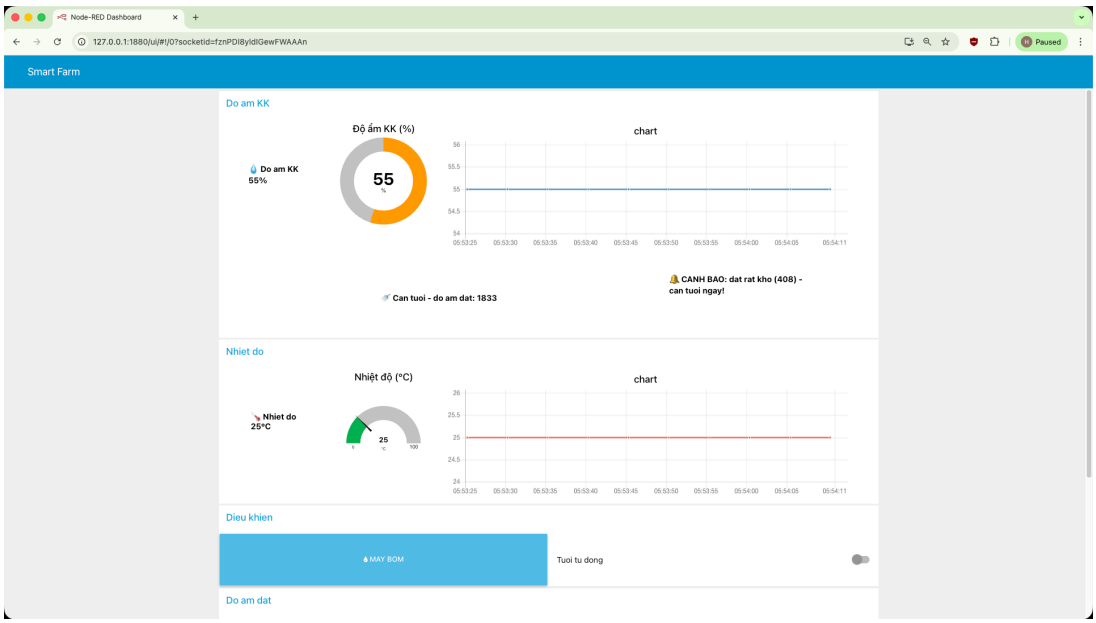
Mọi chương trình xử lý phía máy chủ được tổ chức thành các luồng (flow) trực quan trên Node-RED. Hình 29 thể hiện toàn bộ các node chức năng từ việc tiếp nhận dữ liệu MQTT, giải mã JSON đến việc phân luồng hiển thị và logic điều khiển máy bơm tự động.



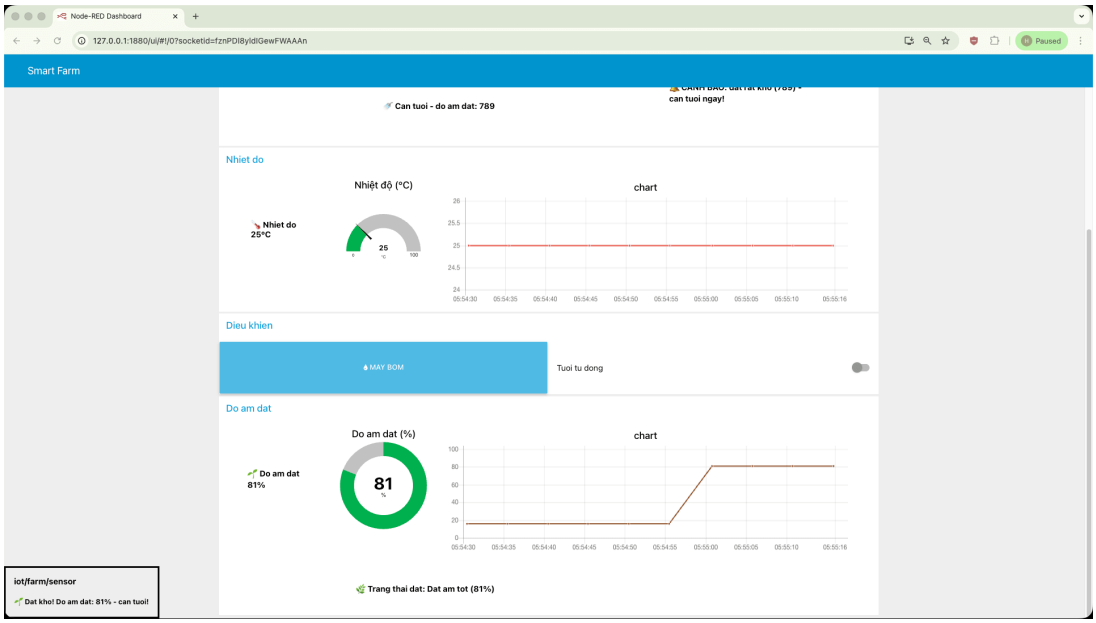
Hình 29: Sơ đồ luồng xử lý dữ liệu (Flow) đầy đủ trên trình biên tập Node-RED

6.4 Giao diện giám sát và điều khiển Dashboard

Kết quả quan trọng nhất của hệ thống là giao diện tương tác người dùng hiện đại và trực quan. Hệ thống Dashboard cung cấp đầy đủ các thông tin cần thiết về môi trường trang trại theo thời gian thực (Hình 30 và Hình 31).



Hình 30: Giao diện Dashboard (Phần 1): Hiển thị đồng hồ đo và các nút điều khiển trung tâm



Hình 31: Giao diện Dashboard (Phần 2): Biểu đồ lịch sử dữ liệu và các thông tin trạng thái đất

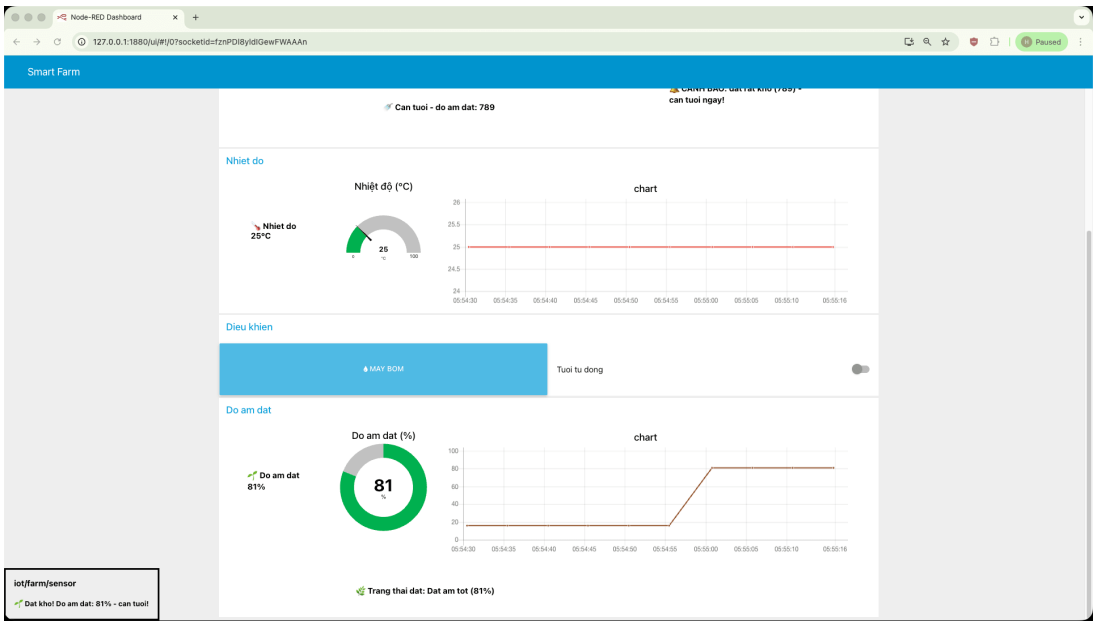
6.5 Phân tích luồng hoạt động thực tế

Hệ thống được thiết kế để xử lý các tình huống môi trường thực tế thông qua các cơ chế phản hồi tự động và cảnh báo trực quan (Hình 32).

Cơ chế cảnh báo: Khi các thông số vượt ngưỡng an toàn, hệ thống sẽ tự động kích hoạt

các chỉ báo:

- **Nhiệt độ:** Khi nhiệt độ > 35°C, Dashboard hiển thị toast cảnh báo nhiệt độ cao.
- **Độ ẩm không khí:** Khi độ ẩm < 40%, hệ thống gợi ý thực hiện tưới nước thông qua thông báo.
- **Độ ẩm đất:** Khi mức ADC < 2048, toast cảnh báo đất khô sẽ xuất hiện và còi Buzzer trên ESP32 phát âm thanh báo động. Đặc biệt, khi độ ẩm giảm sâu (ADC < 1024), Dashboard sẽ hiển thị cảnh báo mức đỏ cực kỳ khô hạn.



Hình 32: Minh họa các thông báo cảnh báo và biểu đồ trạng thái trên giao diện Dashboard

Cơ chế điều khiển: Người dùng có thể thực hiện publish lệnh điều khiển thông qua nút nhấn **Máy bơm** hoặc kích hoạt chế độ **Tưới tự động** để hệ thống tự xử lý dựa trên ngưỡng độ ẩm đất.

Kết quả mô phỏng: Môi trường Wokwi phản hồi chính xác các lệnh MQTT gửi đến; đèn LED (giả lập máy bơm) bật/tắt đúng theo yêu cầu và còi báo động hoạt động đồng bộ với trạng thái cảm biến.

Bảng 2: Tổng hợp các chức năng đã triển khai

Chức năng	**Trạng thái**	**Ghi chú**
Đọc DHT22 mỗi 5 giây	Hoàn thành	GPIO 4, fallback random khi lỗi
Đọc độ ẩm đất (ADC)	Hoàn thành	GPIO 34, potentiometer trên Wokwi
Publish/Subscribe MQTT (TLS)	Hoàn thành	Sử dụng giao thức bảo mật cổng 8883
Hiển thị Gauge nhiệt độ	Hoàn thành	3 vùng màu, min=0, max=100
Hiển thị Gauge độ ẩm KK	Hoàn thành	dạng donut, min=0, max=100
Hiển thị Gauge độ ẩm đất	Hoàn thành	dạng donut, min=0, max=100, 3 vùng màu
Biểu đồ line thời gian thực	Hoàn thành	dot=true, removeOlderUnit=3600
Toast cảnh báo nhiệt độ cao	Hoàn thành	Ngưỡng > 35°C, 8 giây
Toast cảnh báo độ ẩm KK thấp	Hoàn thành	Ngưỡng < 40%, 8 giây
Toast cảnh báo đất khô	Hoàn thành	Ngưỡng soil < 2048, 8 giây
Cảnh báo nguy hiểm (đất rất khô)	Hoàn thành	Ngưỡng soil < 1024, hiển thị đỏ
Bật bơm thủ công (button)	Hoàn thành	Gửi JSON lên iot/farm/control
Tưới tự động (switch)	Hoàn thành	Bật bơm khi ẩm đất < 2048
Cảnh báo buzzer trên ESP32	Hoàn thành	Phát 2kHz/300ms khi `soilRaw < 2048` (firmware)
Reconnect Wi-Fi tự động	Hoàn thành	Kiểm tra trong loop()
Reconnect MQTT non-blocking	Hoàn thành	Thử lại mỗi 5 giây

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Các công việc đã hoàn thành

Qua quá trình triển khai, đề tài đạt các mục tiêu gắn với mã nguồn đính kèm:

Thứ nhất, hệ thống IoT ba tầng hoàn chỉnh: ESP32 + DHT22 + potentiometer (đất) + LED/buzzer; HiveMQ Cloud (TLS); Node-RED Dashboard.

Thứ hai, thu thập và truyền ba thông số mỗi 5 giây qua MQTT (`iot/farm/sensor`), JSON chuẩn.

Thứ ba, Dashboard hiển thị gauge, biểu đồ, toast cảnh báo; điều khiển bơm và tưới tự động qua `iot/farm/control`.

Thứ tư, mô phỏng Wokwi và sơ đồ EasyEDA minh họa thiết kế.

Hệ thống có thể ứng dụng cho nhà kính, ban công, sân thượng.

Ưu điểm: Chi phí thấp; công cụ mã nguồn mở; MQTT và TLS phù hợp IoT; tách luồng Node-RED dễ bảo trì; reconnect MQTT không chặn `loop()`; fallback khi DHT22 trả NaN.

7.2 Công việc chưa thực hiện và hạn chế

Chưa tích hợp máy bơm/relay thật (LED mô phỏng); chưa dùng cảm biến độ ẩm đất chuyên dụng; `setInsecure()` chỉ phù hợp thử nghiệm; chưa lưu lịch sử ngoài biểu đồ Dashboard; thời gian chạy bơm trên firmware cố định `PUMP_DURATION_MS` (chưa dùng `duration` từ JSON để tắt theo giây gửi).

7.3 Hướng phát triển tiếp theo

Thứ nhất, thay potentiometer bằng cảm biến độ ẩm đất capacitive.

Thứ hai, tích hợp relay và máy bơm nước thực tế; nâng cấp firmware để tự động ngắt bơm dựa trên thời gian (duration) nhận từ bản tin JSON.

Thứ ba, xác thực TLS đầy đủ (CA), bỏ `setInsecure()` khi triển khai thực tế.

Thứ tư, InfluxDB/MySQL cho lịch sử và báo cáo.

Thứ năm, ứng dụng di động thay hoặc bổ sung Dashboard.

Thứ sáu, OTA để cập nhật firmware từ xa.

TÀI LIỆU THAM KHẢO

Tài liệu

[1] Espressif Systems. *ESP32 Series Datasheet*. Truy cập: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[2] Node-RED. *Node-RED Documentation*. IBM & others. Truy cập: https://nodered.org/docs/.

[3] HiveMQ. *HiveMQ Cloud*. Truy cập: https://www.hivemq.com/cloud/.

[4] Seeed Studio. *Capacitive Soil Moisture Sensor*. Truy cập: https://wiki.seeedstudio.com/Capacitive_Soil_Moisture_Sensor/.

[5] Adafruit. *DHT sensor library for Arduino*. Truy cập: https://github.com/adafruit/DHT-sensor-library.

[6] Nick O'Leary. *PubSubClient for Arduino*. Truy cập: https://github.com/knolleary/pubsubclient.

[7] Node-RED. *node-red-dashboard*. Truy cập: https://flows.nodered.org/node/node-red-dashboard.

CÁC PHỤ LỤC

PHỤ LỤC A. Mã nguồn chương trình ESP32

Lưu ý: Trước khi nạp firmware, cần cập nhật các thông tin sau trong code: `WIFI_SSID` và `WIFI_PASSWORD` (tên và mật khẩu Wi-Fi); `MQTT_USERNAME` và `MQTT_PASSWORD` (tài khoản HiveMQ Cloud). Thông tin xác thực MQTT đã được ẩn trong tài liệu này vì lý do bảo mật.

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

#ifdef WOKWI
const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASSWORD = "";
#else
const char *WIFI_SSID = "YOUR_WIFI_SSID";
const char *WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
#endif

const char *MQTT_BROKER = "b1c8f2cc5cd4416fb74b671a407dc0ff.s1
    .eu.hivemq.cloud";
const int MQTT_PORT = 8883;
const char *MQTT_CLIENT_ID = "esp32-sensor-01";
const char *MQTT_USERNAME = "hivemq.webclient.1775680760112";
const char *MQTT_PASSWORD = "amy&f0VPG3c5<8,U>qAB";

const char *TOPIC_SENSOR = "iot/farm/sensor";
const char *TOPIC_CONTROL = "iot/farm/control";

#define DHT_PIN 4
#define DHT_TYPE DHT22
#define LED_PIN 5
#define SOIL_PIN 34
#define BUZZER_PIN 18
```

```
DHT dht(DHT_PIN, DHT_TYPE);
WiFiClientSecure espClient;
PubSubClient mqttClient(espClient);

unsigned long lastPublish = 0;
unsigned long lastMQTTRetry = 0;
const unsigned long PUBLISH_INTERVAL = 5000;
const unsigned long MQTT_RETRY_MS = 5000;

bool pumpActive = false;
unsigned long pumpStartTime = 0;
const unsigned long PUMP_DURATION_MS = 5000;

void toneBuzzer(int pin, int freq, int durationMs)
{
  ledcSetup(0, freq, 8);
  ledcAttachPin(pin, 0);
  ledcWrite(0, 127);
  delay(durationMs);
  ledcWrite(0, 0);
  ledcDetachPin(pin);
}

void stopPump()
{
  digitalWrite(LED_PIN, LOW);
  pumpActive = false;
}

void handleControlPayload(const char *payload, size_t len)
{
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, payload, len
   );
  if (err)
    return;

  if (doc.containsKey("pump"))
  {
    bool pumpCmd = doc["pump"].as<bool>();
```

```
    if (pumpCmd)
    {
      int durationS = doc.containsKey("duration") ? doc["duration"].as<int>() : 5;
      pumpActive = true;
      pumpStartTime = millis();
      digitalWrite(LED_PIN, HIGH);
      Serial.printf("[PUMP] Bat bom trong %d giay\n", durationS);
    }
    else
    {
      stopPump();
      Serial.println("[PUMP] Tat bom");
    }
  }
}

void mqttCallback(char *topic, byte *payload, unsigned int length)
{
  if (strcmp(topic, TOPIC_CONTROL) == 0)
  {
    char msg[length + 1];
    memcpy(msg, payload, length);
    msg[length] = '\0';
    handleControlPayload(msg, length);
  }
}

void setupWiFi()
{
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(100);
```

```
    Serial.print(".");
  }

  Serial.println();
  Serial.print("WiFi connected! IP: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT()
{
  Serial.print("[MQTT] Connecting...");
  if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USERNAME,
   MQTT_PASSWORD))
  {
    Serial.println(" Connected!");
    mqttClient.subscribe(TOPIC_CONTROL);
    Serial.printf("[MQTT] Subscribed: %s\n", TOPIC_CONTROL);
  }
  else
  {
    Serial.printf(" Failed, error: %d\n", mqttClient.state());
  }
}

void publishSensorData(float temp, float hum, int soilRaw)
{
  StaticJsonDocument<512> doc;
  doc["device"] = "ESP32-01";
  doc["temperature"] = temp;
  doc["humidity"] = hum;
  doc["soil"] = soilRaw;
  doc["timestamp"] = millis();

  char buffer[512];
  size_t n = serializeJson(doc, buffer);
  mqttClient.publish(TOPIC_SENSOR, buffer, n);

  Serial.printf("Published: Temp=%.1f C, Hum=%.1f %%, Soil=%d\
   n", temp, hum, soilRaw);
}

```

```
void readSensor(float &temp, float &hum, int &soilRaw)
{
  temp = dht.readTemperature();
  hum = dht.readHumidity();
  soilRaw = analogRead(SOIL_PIN);

  if (isnan(temp))
  {
    temp = random(200, 350) / 10.0;
  }
  if (isnan(hum))
  {
    hum = random(400, 800) / 10.0;
  }
}

void checkPumpTimeout()
{
  if (pumpActive && (millis() - pumpStartTime >=
   PUMP_DURATION_MS))
  {
    stopPump();
    Serial.println("[PUMP] Het thoi gian, tat bom");
  }
}

void setup()
{
  Serial.begin(115200);
  Serial.println("\n========================================")
   ;
  Serial.println("  ESP32 - Smart Farm IoT System");
  Serial.println("========================================");

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  pinMode(SOIL_PIN, INPUT);
```

```
  dht.begin();

  espClient.setInsecure();

  setupWiFi();

  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(mqttCallback);

  Serial.println("Setup complete!");
}

void loop()
{
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("[WiFi] Lost connection, reconnecting...");
    setupWiFi();
  }

  if (!mqttClient.connected())
  {
    if (millis() - lastMQTTRetry >= MQTT_RETRY_MS)
    {
      lastMQTTRetry = millis();
      reconnectMQTT();
    }
  }
  mqttClient.loop();

  checkPumpTimeout();

  unsigned long now = millis();

  if (now - lastPublish >= PUBLISH_INTERVAL)
  {
    lastPublish = now;

    float temperature, humidity;
    int soilRaw;
```

```
    readSensor(temperature, humidity, soilRaw);

    Serial.println("\n--- Sensor Data ---");
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");
    Serial.print("Soil Moisture (raw): ");
    Serial.println(soilRaw);

    if (soilRaw < 2048)
    {
      toneBuzzer(BUZZER_PIN, 2000, 300);
      Serial.println("[ALERT] Dat khong hop li - can tuoi!");
    }

    if (mqttClient.connected())
    {
      publishSensorData(temperature, humidity, soilRaw);
    }
    else
    {
      Serial.println("[MQTT] Not connected, skipping...");
    }
  }
}
```

Listing 1: Firmware ESP32 — main.cpp

PHỤ LỤC B. Mã nguồn Node-RED (JSON Flow)

Các bước import flow vào Node-RED:

1. Truy cập Node-RED tại `http://127.0.0.1:1880`.
2. Nhấn Menu → Import.
3. Chọn thẻ Clipboard, paste toàn bộ nội dung JSON bên dưới.
4. Nhấn **Import** để thêm flow.

5. Nhấn **Deploy** (nút đỏ góc phải) để chạy flow.

Lưu ý: Sau khi import, kiểm tra và cập nhật cấu hình node **mqtt-broker**: hostname, port (8883), username, password phải khớp với tài khoản HiveMQ Cloud đang sử dụng.


```
[
  {
    "id": "bd7b79c3fbf5417a",
    "type": "mqtt in",
    "z": "28972163d1638f7e",
    "name": "Nhan cam bien",
    "topic": "iot/farm/sensor",
    "qos": "0",
    "datatype": "json",
    "broker": "d4179fb10b941343",
    "nl": false,
    "rap": false,
    "inputs": 0,
    "x": 260,
    "y": 380,
    "wires": [
      [
        "a31199f64a6ad255",
        "b8e99199386b9667",
        "01e7a8e29907dd5c",
        "e57427a9f993e160",
        "b91420f0802c3665",
        "b24a110f89916cc1",
        "f8a1gaugefn01",
        "f8a1gaugefn02",
        "9e2f992ea37b88e9",
        "ef65ccde2a9dc727",
        "soilfuncfn01",
        "soilgaugefn01",
        "soilchartfn01",
        "soiltextfn01"
      ]
    ]
  },
  {
    "id": "f8a1gaugefn01",
    "type": "function",
```

```
    "z": "28972163d1638f7e",
    "name": "Gauge: nhiet do",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar t = parseFloat(p.temperature);\nif (isNaN(t)) return null;\nmsg.payload = t;\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 500,
    "y": 400,
    "wires": [
      [
        "f8a1gaugeui01"
      ]
    ]
  },
  {
    "id": "f8a1gaugefn02",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Gauge: do am KK",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar h = parseFloat(p.humidity);\nif (isNaN(h)) return null;\nmsg.payload = h;\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 500,
    "y": 440,
    "wires": [
      [
```

```
        "f8a1gaugeui02"
      ]
    ]
  },
  {
    "id": "f8a1gaugeui01",
    "type": "ui_gauge",
    "z": "28972163d1638f7e",
    "name": "Gauge nhiet do",
    "group": "f1e2d3c4b5a69708",
    "order": 2,
    "width": "4",
    "height": "4",
    "gtype": "gage",
    "title": "Nhiet do (C)",
    "label": "C",
    "format": "{{payload}}",
    "min": 0,
    "max": 100,
    "colors": [
      "#00b050",
      "#e6e600",
      "#ca3838"
    ],
    "seg1": "33",
    "seg2": "67",
    "diff": false,
    "className": "",
    "x": 720,
    "y": 400,
    "wires": []
  },
  {
    "id": "f8a1gaugeui02",
    "type": "ui_gauge",
    "z": "28972163d1638f7e",
    "name": "Gauge do am KK",
    "group": "d2ca79a83590b557",
    "order": 2,
    "width": "4",
    "height": "4",
```

```
    "gtype": "donut",
    "title": "Do am KK (%)",
    "label": "%",
    "format": "{{payload}}",
    "min": 0,
    "max": 100,
    "colors": [
      "#00b050",
      "#ff9900",
      "#ca3838"
    ],
    "seg1": "33",
    "seg2": "67",
    "diff": false,
    "className": "",
    "x": 720,
    "y": 440,
    "wires": []
  },
  {
    "id": "b91420f0802c3665",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Chart: nhiet do (so)",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar t = parseFloat(p.temperature);\nif (isNaN(t)) return null;\nmsg.payload = t;\nmsg.topic = 'Nhiet do C';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 480,
    "wires": [
      [
        "cc5a9fcda823fb39"
      ]
```

```
    ]
  },
  {
    "id": "b24a110f89916cc1",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Chart: do am KK (so)",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar h = parseFloat(p.humidity);\nif (isNaN(h)) return null;\nmsg.payload = h;\nmsg.topic = 'Do am KK %';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 520,
    "wires": [
      [
        "2c3a97d97ea9da30"
      ]
    ]
  },
  {
    "id": "a31199f64a6ad255",
    "type": "switch",
    "z": "28972163d1638f7e",
    "name": "Nhiet do > 35C",
    "property": "payload.temperature",
    "propertyType": "msg",
    "rules": [
      {
        "t": "gt",
        "v": "35",
        "vt": "num"
      }
    ],
    "checkall": "true",
```

```
    "repair": false,
    "outputs": 1,
    "x": 540,
    "y": 220,
    "wires": [
      [
        "f9a1toastfn01"
      ]
    ]
  },
  {
    "id": "f9a1toastfn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Toast text: nhiet",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nvar t = p && p.temperature != null ? Number(p.temperature) : NaN;\nif (isNaN(t)) return null;\nmsg.payload = 'Nhiet do cao: ' + t + 'C';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 760,
    "y": 220,
    "wires": [
      [
        "425f5a8bf7c2a0b5"
      ]
    ]
  },
  {
    "id": "425f5a8bf7c2a0b5",
    "type": "ui_toast",
    "z": "28972163d1638f7e",
    "position": "bottom left",
    "displayTime": "8",
    "outputs": 0,
```

```
    "className": "",
    "name": "Thong bao nhiet do",
    "x": 990,
    "y": 220,
    "wires": []
  },
  {
    "id": "b8e99199386b9667",
    "type": "switch",
    "z": "28972163d1638f7e",
    "name": "Do am KK < 40%",
    "property": "payload.humidity",
    "propertyType": "msg",
    "rules": [
      {
        "t": "lt",
        "v": "40",
        "vt": "num"
      }
    ],
    "checkall": "true",
    "repair": false,
    "outputs": 1,
    "x": 540,
    "y": 300,
    "wires": [
      [
        "f9a1toastfn02",
        "dbb4a8c895814460"
      ]
    ]
  },
  {
    "id": "f9a1toastfn02",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Toast text: am KK",
    "func": "var p = msg.payload;\nif (typeof p === 'string')
 { try { p = JSON.parse(p); } catch (e) { return null; } }\
 nvar h = p && p.humidity != null ? Number(p.humidity) : NaN
 ;\nif (isNaN(h)) return null;\nmsg.payload = 'Do am KK thap
```

```
: ' + h + '% - can nhac tuoi';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 760,
    "y": 300,
    "wires": [
      [
        "e9272ea0978a6ed1"
      ]
    ]
  },
  {
    "id": "e9272ea0978a6ed1",
    "type": "ui_toast",
    "z": "28972163d1638f7e",
    "position": "bottom left",
    "displayTime": "8",
    "outputs": 0,
    "className": "",
    "name": "Thong bao dat kho",
    "x": 990,
    "y": 300,
    "wires": []
  },
  {
    "id": "dbb4a8c895814460",
    "type": "switch",
    "z": "28972163d1638f7e",
    "name": "Che do tuoi tu dong",
    "property": "manual",
    "propertyType": "flow",
    "rules": [
      {
        "t": "eq",
        "v": "true",
        "vt": "bool"
      }
```

```
    ],
    "checkall": "true",
    "repair": false,
    "outputs": 1,
    "x": 780,
    "y": 340,
    "wires": [
      [
        "df4e79c154d58a50"
      ]
    ]
  },
  {
    "id": "df4e79c154d58a50",
    "type": "change",
    "z": "28972163d1638f7e",
    "name": "Bat bom 5s",
    "rules": [
      {
        "t": "set",
        "p": "payload",
        "pt": "msg",
        "to": "{\"pump\":true,\"duration\":5}",
        "tot": "json"
      }
    ],
    "action": "",
    "property": "",
    "from": "",
    "to": "",
    "reg": false,
    "x": 1070,
    "y": 460,
    "wires": [
      [
        "52d4228c218a253a"
      ]
    ]
  },
  {
    "id": "01e7a8e29907dd5c",
```

```
    "type": "switch",
    "z": "28972163d1638f7e",
    "name": "Do am dat < 2048",
    "property": "payload.soil",
    "propertyType": "msg",
    "rules": [
      {
        "t": "lt",
        "v": "2048",
        "vt": "num"
      }
    ],
    "checkall": "true",
    "repair": false,
    "outputs": 1,
    "x": 540,
    "y": 600,
    "wires": [
      [
        "c90822af4a63eb54",
        "soilalertfn01"
      ]
    ]
  },
  {
    "id": "c90822af4a63eb54",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Hien trang bom",
    "func": "var s = msg.payload.soil;\nmsg.payload = 'Can tuoi - do am dat: ' + (s != null ? s : '?') + '';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 740,
    "y": 600,
    "wires": [
```

```
      [
        "22c86325ecbf70be"
      ]
    ]
  },
  {
    "id": "22c86325ecbf70be",
    "type": "ui_text",
    "z": "28972163d1638f7e",
    "group": "d2ca79a83590b557",
    "order": 4,
    "width": "16",
    "height": "3",
    "name": "Trang thai tuoi",
    "label": "",
    "format": "<span style=\"font-size:15px;line-height
:1.4\">{{msg.payload}}</span>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#1b5e20",
    "x": 980,
    "y": 600,
    "wires": []
  },
  {
    "id": "e57427a9f993e160",
    "type": "switch",
    "z": "28972163d1638f7e",
    "name": "Do am dat < 1024",
    "property": "payload.soil",
    "propertyType": "msg",
    "rules": [
      {
        "t": "lt",
        "v": "1024",
        "vt": "num"
      }
    ],
```

```
    "checkall": "true",
    "repair": false,
    "outputs": 1,
    "x": 500,
    "y": 660,
    "wires": [
      [
        "673687e437ae8622"
      ]
    ]
  },
  {
    "id": "673687e437ae8622",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Hien canh bao",
    "func": "var s = msg.payload.soil;\nmsg.payload = 'CANH BAO: dat rat kho (' + (s != null ? s : '?') + ') - can tuoi ngay!';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 740,
    "y": 660,
    "wires": [
      [
        "ac643c3a74928b73"
      ]
    ]
  },
  {
    "id": "ac643c3a74928b73",
    "type": "ui_text",
    "z": "28972163d1638f7e",
    "group": "d2ca79a83590b557",
    "order": 5,
    "width": "6",
    "height": "2",
```

```
    "name": "Canh bao Buzzer",
    "format": "<span style=\"font-size:15px;line-height:1.4;color:#b71c1c;font-weight:600\">{{msg.payload}}</span>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "",
    "x": 990,
    "y": 660,
    "wires": []
  },
  {
    "id": "soilfuncfn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Soil: hien thi so",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar s = parseFloat(p.soil);\nif (isNaN(s)) return null;\nmsg.payload = Math.round((1 - s / 4095) * 100);\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 820,
    "wires": [
      [
        "soildisplay01"
      ]
    ]
  },
  {
    "id": "soildisplay01",
    "type": "ui_text",
    "z": "28972163d1638f7e",
```

```
    "group": "5e4f3c2d1a0b9876",
    "order": 1,
    "width": "4",
    "height": "4",
    "name": "Do am dat",
    "label": "",
    "format": "<div style=\"display:flex;align-items:center;justify-content:center;height:100%;min-height:120px\"><div style=\"text-align:center\"><div style=\"font-size:14px;opacity:0.85;margin-bottom:8px\">Do am dat</div><span style=\"font-size:36px;font-weight:700;line-height:1.1\">{{msg.payload}}<span style=\"font-size:22px\">%</span></span></div></div>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#000000",
    "x": 720,
    "y": 820,
    "wires": []
  },
  {
    "id": "soilgaugefn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Soil: gauge",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar s = parseFloat(p.soil);\nif (isNaN(s)) return null;\nmsg.payload = Math.round((1 - s / 4095) * 100);\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 880,
```

```
    "wires": [
      [
        "soilgaugeui01"
      ]
    ]
  },
  {
    "id": "soilgaugeui01",
    "type": "ui_gauge",
    "z": "28972163d1638f7e",
    "name": "Gauge do am dat",
    "group": "5e4f3c2d1a0b9876",
    "order": 2,
    "width": "4",
    "height": "4",
    "gtype": "donut",
    "title": "Do am dat (%)",
    "label": "%",
    "format": "{{payload}}",
    "min": 0,
    "max": 100,
    "colors": [
      "#ca3838",
      "#ff9900",
      "#00b050"
    ],
    "seg1": "33",
    "seg2": "67",
    "diff": false,
    "className": "",
    "x": 720,
    "y": 880,
    "wires": []
  },
  {
    "id": "soilchartfn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Soil: chart (so)",
    "func": "var p = msg.payload;\nif (typeof p === 'string')
 { try { p = JSON.parse(p); } catch (e) { return null; } }\
```

```
nif (!p || typeof p !== 'object') return null;\nvar s =
parseFloat(p.soil);\nif (isNaN(s)) return null;\nmsg.
payload = Math.round((1 - s / 4095) * 100);\nmsg.topic = '
Do am dat %';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 940,
    "wires": [
      [
        "soilchartui01"
      ]
    ]
  },
  {
    "id": "soilchartui01",
    "type": "ui_chart",
    "z": "28972163d1638f7e",
    "name": "Bieu do do am dat",
    "group": "5e4f3c2d1a0b9876",
    "order": 3,
    "width": "15",
    "height": "5",
    "label": "chart",
    "chartType": "line",
    "legend": "false",
    "xformat": "HH:mm:ss",
    "interpolate": "linear",
    "nodata": "",
    "dot": true,
    "ymin": "",
    "ymax": "",
    "removeOlder": 1,
    "removeOlderPoints": "10",
    "removeOlderUnit": "3600",
    "cutout": 0,
    "useOneColor": false,
```

```
    "useUTC": false,
    "colors": [
      "#8b4513",
      "#aec7e8",
      "#2ca02c",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 720,
    "y": 940,
    "wires": [
      []
    ]
  },
  {
    "id": "soilalertfn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Toast: dat kho",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nvar s = p && p.soil != null ? Number(p.soil) : NaN;\nif (isNaN(s)) return null;\nmsg.payload = 'Dat kho! Do am dat: ' + Math.round((1 - s / 4095) * 100) + '% - can tuoi!';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 780,
    "y": 760,
    "wires": [
```

```
      [
        "soiltoastui01"
      ]
    ]
  },
  {
    "id": "soiltoastui01",
    "type": "ui_toast",
    "z": "28972163d1638f7e",
    "position": "bottom left",
    "displayTime": "8",
    "outputs": 0,
    "class Name": "",
    "name": "Thong bao dat kho",
    "x": 990,
    "y": 760,
    "wires": []
  },
  {
    "id": "soiltextfn01",
    "type": "function",
    "z": "28972163d1638f7e",
    "name": "Soil: trang thai",
    "func": "var p = msg.payload;\nif (typeof p === 'string') { try { p = JSON.parse(p); } catch (e) { return null; } }\nif (!p || typeof p !== 'object') return null;\nvar s = parseFloat(p.soil);\nif (isNaN(s)) return null;\nvar pct = Math.round((1 - s / 4095) * 100);\nvar status = pct < 30 ? 'Do am thap - can tuoi!' : (pct < 60 ? 'Do am trung binh' : 'Dat am tot');\nmsg.payload = 'Trang thai dat: ' + status + ' (' + pct + '%)';\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 510,
    "y": 1000,
    "wires": [
      [
```

```
        "soilstatusui01"
      ]
    ]
  },
  {
    "id": "soilstatusui01",
    "type": "ui_text",
    "z": "28972163d1638f7e",
    "group": "5e4f3c2d1a0b9876",
    "order": 4,
    "width": "15",
    "height": "2",
    "name": "Trang thai dat",
    "label": "",
    "format": "<span style=\"font-size:14px;line-height
:1.4\">{{msg.payload}}</span>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#2e7d32",
    "x": 720,
    "y": 1000,
    "wires": []
  },
  {
    "id": "cc5a9fcda823fb39",
    "type": "ui_chart",
    "z": "28972163d1638f7e",
    "name": "Bieu do nhiet do",
    "group": "f1e2d3c4b5a69708",
    "order": 3,
    "width": "15",
    "height": "5",
    "label": "chart",
    "chartType": "line",
    "legend": "false",
    "xformat": "HH:mm:ss",
    "interpolate": "linear",
    "nodata": "",
```

```
    "dot": true,
    "ymin": "",
    "ymax": "",
    "removeOlder": 1,
    "removeOlderPoints": "10",
    "removeOlderUnit": "3600",
    "cutout": 0,
    "useOneColor": false,
    "useUTC": false,
    "colors": [
      "#e74c3c",
      "#aec7e8",
      "#ff7f0e",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 720,
    "y": 480,
    "wires": [
      []
    ]
  },
  {
    "id": "2c3a97d97ea9da30",
    "type": "ui_chart",
    "z": "28972163d1638f7e",
    "name": "Bieu do do am KK",
    "group": "d2ca79a83590b557",
    "order": 3,
    "width": "15",
    "height": "5",
    "label": "chart",
    "chartType": "line",
    "legend": "false",
```

```
    "xformat": "HH:mm:ss",
    "interpolate": "linear",
    "nodata": "",
    "dot": true,
    "ymin": "",
    "ymax": "",
    "removeOlder": 1,
    "removeOlderPoints": "10",
    "removeOlderUnit": "3600",
    "cutout": 0,
    "useOneColor": false,
    "useUTC": false,
    "colors": [
      "#2980b9",
      "#aec7e8",
      "#2ca02c",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000",
      "#000000"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 720,
    "y": 520,
    "wires": [
      []
    ]
  },
  {
    "id": "9e2f992ea37b88e9",
    "type": "ui_text",
    "z": "28972163d1638f7e",
    "group": "f1e2d3c4b5a69708",
    "order": 1,
    "width": "4",
    "height": "4",
    "name": "Nhiet do",
```

```
    "label": "",
    "format": "<div style=\"display:flex;align-items:center;justify-content:center;height:100%;min-height:120px\"><div style=\"text-align:center\"><div style=\"font-size:14px;opacity:0.85;margin-bottom:8px\">Nhiet do</div><span style=\"font-size:36px;font-weight:700;line-height:1.1\">{{msg.payload.temperature}}<span style=\"font-size:22px\">C</span></span></div></div>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#000000",
    "x": 520,
    "y": 160,
    "wires": []
  },
  {
    "id": "ef65ccde2a9dc727",
    "type": "ui_text",
    "z": "28972163d1638f7e",
    "group": "d2ca79a83590b557",
    "order": 1,
    "width": "4",
    "height": "4",
    "name": "Do am KK",
    "label": "",
    "format": "<div style=\"display:flex;align-items:center;justify-content:center;height:100%;min-height:120px\"><div style=\"text-align:center\"><div style=\"font-size:14px;opacity:0.85;margin-bottom:8px\">Do am KK</div><span style=\"font-size:36px;font-weight:700;line-height:1.1\">{{msg.payload.humidity}}<span style=\"font-size:22px\">%</span></span></div></div>",
    "layout": "row-center",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#000000",
```

```
    "x": 510,
    "y": 100,
    "wires": []
  },
  {
    "id": "9782277746682320",
    "type": "ui_button",
    "z": "28972163d1638f7e",
    "name": "Bat/Tat bom",
    "group": "22743ae4eedbebe6",
    "order": 1,
    "width": "12",
    "height": "2",
    "passthru": false,
    "label": "May bom",
    "tooltip": "",
    "color": "#ffffff",
    "bgcolor": "",
    "className": "",
    "icon": "fa-tint",
    "payload": "{\"pump\":true,\"duration\":5}",
    "payloadType": "json",
    "topic": "iot/farm/control",
    "topicType": "str",
    "x": 230,
    "y": 700,
    "wires": [
      [
        "52d4228c218a253a"
      ]
    ]
  },
  {
    "id": "9269adf2dde7a036",
    "type": "ui_switch",
    "z": "28972163d1638f7e",
    "name": "Tuoi tu dong",
    "label": "Tuoi tu dong",
    "tooltip": "",
    "group": "22743ae4eedbebe6",
    "order": 2,
```

```
    "width": "12",
    "height": "2",
    "passthru": true,
    "decouple": "false",
    "topic": "",
    "topicType": "str",
    "onvalue": "",
    "onvalueType": "str",
    "onicon": "",
    "oncolor": "",
    "offvalue": "",
    "offvalueType": "str",
    "officon": "",
    "offcolor": "",
    "animate": true,
    "className": "",
    "x": 240,
    "y": 760,
    "wires": [
      [
        "f0dd0c472d5d7e77"
      ]
    ]
  },
  {
    "id": "f0dd0c472d5d7e77",
    "type": "change",
    "z": "28972163d1638f7e",
    "name": "Set flow.manual",
    "rules": [
      {
        "t": "set",
        "p": "manual",
        "pt": "flow",
        "to": "payload",
        "tot": "msg"
      }
    ],
    "action": "",
    "property": "",
    "from": "",
```

```
    "to": "",
    "reg": false,
    "x": 440,
    "y": 760,
    "wires": [
      []
    ]
  },
  {
    "id": "52d4228c218a253a",
    "type": "mqtt out",
    "z": "28972163d1638f7e",
    "name": "Gui dieu khien bom",
    "topic": "iot/farm/control",
    "qos": "0",
    "retain": "false",
    "broker": "d4179fb10b941343",
    "x": 1420,
    "y": 700,
    "wires": []
  },
  {
    "id": "d4179fb10b941343",
    "type": "mqtt-broker",
    "name": "Smart",
    "broker": "b1c8f2cc5cd4416fb74b671a407dc0ff.s1.eu.hivemq.
 cloud",
    "port": "8883",
    "tls": "",
    "clientid": "",
    "autoConnect": true,
    "usetls": true,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
```

```
    "closeTopic": "",
    "closeQos": "0",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  },
  {
    "id": "f1e2d3c4b5a69708",
    "type": "ui_group",
    "name": "Nhiet do",
    "tab": "2ec7089e7e446577",
    "order": 2,
    "disp": true,
    "width": "24",
    "collapse": false,
    "className": ""
  },
  {
    "id": "d2ca79a83590b557",
    "type": "ui_group",
    "name": "Do am KK",
    "tab": "2ec7089e7e446577",
    "order": 1,
    "disp": true,
    "width": "24",
    "collapse": false,
    "className": ""
  },
  {
    "id": "22743ae4eedbebe6",
    "type": "ui_group",
    "name": "Dieu khien",
    "tab": "2ec7089e7e446577",
    "order": 3,
```

```
    "disp": true,
    "width": "24",
    "collapse": false,
    "className": ""
  },
  {
    "id": "5e4f3c2d1a0b9876",
    "type": "ui_group",
    "name": "Do am dat",
    "tab": "2ec7089e7e446577",
    "order": 4,
    "disp": true,
    "width": "24",
    "collapse": false,
    "className": ""
  },
  {
    "id": "2ec7089e7e446577",
    "type": "ui_tab",
    "name": "Smart Farm",
    "icon": "fa-leaf",
    "order": 1
  },
  {
    "id": "b71f21b1ed76d1c3",
    "type": "global-config",
    "env": [],
    "modules": {
      "node-red-dashboard": "3.6.6"
    }
  }
]
```


Listing 2: Node-RED Flow (JSON) — node-red.json

PHỤ LỤC C. Mã nguồn sơ đồ Wokwi (JSON)

Các bước mở sơ đồ mạch trên Wokwi:

1. Truy cập https://wokwi.com/.

2. Đăng nhập (hoặc tạo tài khoản mới).

3. Tạo project mới hoặc import file JSON bên dưới.

4. Nhấn Play để chạy mô phỏng.

```json
{
  "version": 1,
  "author": "Smart Farm",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht1",
      "top": -20,
      "left": 240,
      "attrs": {
        "temperature": "25",
        "humidity": "55"
      }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": 80,
      "left": -120,
      "attrs": {
        "color": "green"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_dht",
      "top": -20,
      "left": 160,
```

```
      "attrs": {
        "value": "10000"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_led",
      "top": 80,
      "left": -200,
      "attrs": {
        "value": "220"
      }
    },
    {
      "type": "wokwi-potentiometer",
      "id": "pot1",
      "top": 200,
      "left": -120,
      "attrs": {
        "value": "512"
      }
    },
    {
      "type": "wokwi-buzzer",
      "id": "buzzer1",
      "top": 80,
      "left": 200,
      "attrs": {
        "frequency": "2000"
      }
    }
  ],
  "connections": [
    ["esp:TX", "$serialMonitor:RX", "", []],
    ["esp:RX", "$serialMonitor:TX", "", []],

    ["esp:3V3", "dht1:VCC", "red", []],
    ["esp:GND.2", "dht1:GND", "black", []],
    ["esp:3V3", "r_dht:1", "red", []],
    ["r_dht:2", "dht1:SDA", "orange", []],
    ["dht1:SDA", "esp:4", "yellow", []],
```

```
    ["esp:5", "r_led:1", "green", []],
    ["r_led:2", "led1:A", "green", []],
    ["esp:GND.2", "led1:C", "black", []],

    ["esp:3V3", "pot1:VCC", "red", []],
    ["esp:GND.2", "pot1:GND", "black", []],
    ["pot1:SIG", "esp:34", "blue", []],

    ["esp:GND.2", "buzzer1:1", "black", []],
    ["esp:18", "buzzer1:2", "purple", []]
  ],
  "dependencies": {}
}
```

Listing 3: Sơ đồ mạch Wokwi (JSON) — diagram.json